

TRƯỜNG ĐẠI HỌC / CAO ĐẲNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN
XÂY DỰNG WEBSITE BÁN THỰC PHẨM
VÀ TÍCH HỢP HỆ THỐNG GỢI Ý LẠI
(HYBRID RECOMMENDATION SYSTEM)



Sinh viên thực hiện: Trần Phan Hoàng Triều

Mã sinh viên: 3122411226

Lớp: DCT122C2

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Thành phố Hồ Chí Minh, 15/04/2026

LỜI CAM ĐOAN

Em xin cam đoan báo cáo đồ án này là kết quả của quá trình tìm hiểu, phân tích và xây dựng hệ thống dựa trên yêu cầu đề tài. Các nội dung trình bày trong báo cáo được biên soạn nhằm mô tả đầy đủ bài toán, cơ sở lý thuyết, phương pháp thiết kế, quá trình triển khai và kết quả đạt được của website bán thực phẩm có tích hợp hệ thống gợi ý lai. Những đoạn mã, mô hình, bảng biểu và hình minh họa được sử dụng với mục đích học thuật, phục vụ cho việc diễn giải giải pháp kỹ thuật trong đồ án.

Trong quá trình hoàn thiện, báo cáo có kế thừa kiến thức nền tảng từ các tài liệu về thương mại điện tử, Flask, cơ sở dữ liệu quan hệ và hệ thống gợi ý. Em xin chịu trách nhiệm về tính trung thực của nội dung trình bày và cam kết không sử dụng báo cáo này cho mục đích vi phạm quy định học thuật.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã định hướng đề tài, góp ý phương pháp thực hiện và hỗ trợ em trong quá trình hoàn thiện đồ án. Những nhận xét về phạm vi hệ thống, cách tổ chức chức năng và phương pháp đánh giá thuật toán đã giúp em xây dựng báo cáo có cấu trúc rõ ràng hơn.

Em cũng xin cảm ơn thầy cô trong khoa đã cung cấp nền tảng kiến thức về lập trình web, cơ sở dữ liệu, phân tích thiết kế hệ thống và trí tuệ nhân tạo. Đây là cơ sở quan trọng để em có thể kết hợp giữa một ứng dụng thương mại điện tử thực tế với module gợi ý sản phẩm dựa trên dữ liệu hành vi người dùng.

TÓM TẮT ĐỒ ÁN

Đồ án tập trung xây dựng một website bán thực phẩm trực tuyến bằng Python Flask, kết hợp hệ thống gợi ý lai nhằm cá nhân hóa trải nghiệm mua sắm. Hệ thống cho phép khách hàng đăng ký, đăng nhập, duyệt sản phẩm theo danh mục, thêm sản phẩm vào giỏ hàng, đặt hàng và theo dõi trạng thái đơn. Ở phía quản trị, admin có thể quản lý danh mục, sản phẩm, tồn kho, hình ảnh, đơn hàng và trạng thái xử lý.

Điểm nhấn của đồ án là module Hybrid Recommendation System, kết hợp Content-Based Filtering và Collaborative Filtering. Content-Based Filtering khai thác nội dung sản phẩm như tên, mô tả, danh mục để tính độ tương tự bằng TF-IDF và Cosine Similarity. Collaborative Filtering khai thác lịch sử đơn hàng để tìm các người dùng có hành vi tương đồng. Kết quả cuối cùng được kết hợp bằng công thức có trọng số, đồng thời bổ sung cơ chế xử lý Cold Start cho người dùng mới hoặc sản phẩm mới.

Báo cáo trình bày đầy đủ bối cảnh đề tài, mục tiêu, phạm vi, cơ sở lý thuyết, phân tích yêu cầu, thiết kế cơ sở dữ liệu, kiến trúc hệ thống, thuật toán gợi ý, triển khai chức năng, kiểm thử, đánh giá, hạn chế và hướng phát triển.

MỤC LỤC

Contents

LỜI CAM ĐOAN.....	3
LỜI CẢM ƠN.....	4
TÓM TẮT ĐỒ ÁN	5
MỤC LỤC	6
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	10
1.1 Bối cảnh và lý do chọn đề tài	10
1.2 Mục tiêu nghiên cứu	11
1.3 Đối tượng và phạm vi đề tài	12
1.4 Kiến trúc tổng quan	13
1.5 Ý nghĩa thực tiễn	14
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	15
2.1 Tổng quan về thương mại điện tử thực phẩm.....	15
2.2 Hệ thống gợi ý	15
2.3 Content-Based Filtering.....	16
2.4 Collaborative Filtering.....	17
2.5 Hybrid Recommendation.....	17
2.6 Mô hình dữ liệu (ERD).....	18
2.7 Luồng nghiệp vụ chính của người dùng.....	21
2.8 Công nghệ sử dụng	22
2.9 Vibe Engineering và GenAI trong phát triển phần mềm.....	22
2.9.1 Workflow dùng AI trong development pipeline	22
2.9.2 Vai trò của AI trong development pipeline	24
2.8.3 Minh họa Prompt → Code sinh ra → Chỉnh sửa	25
2.8.4 Đánh giá hiệu quả sử dụng AI.....	26
CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT	27

3.1 Tổng quan phương pháp đề xuất	27
3.2 Thiết kế cơ sở dữ liệu (RDBMS qua SQLAlchemy)	28
3.2.1. Mô tả chi tiết từng bảng và ý nghĩa các trường.....	29
3.2.2. Các ràng buộc, chỉ mục (index) và tối ưu truy vấn	30
3.3. Phương pháp xây dựng hệ gợi ý lai (Hybrid Recommendation)	31
3.3.1. Bước 1 — Huấn luyện Content-Based Filtering (TF-IDF + Cosine Similarity)	32
3.3.2. Bước 2 — Huấn luyện Collaborative Filtering (User-User Similarity, Top K=10).....	35
3.3.3. Bước 3 — Kết hợp Hybrid và xử lý Cold Start.....	36
3.3.4. So sánh phương pháp đề xuất với các phương pháp khác	39
3.3.5. Phân tích độ phức tạp tính toán	40
3.4. Phương pháp triển khai kỹ thuật.....	41
3.5.2. Đánh giá hiệu quả áp dụng AI trong quá trình phát triển.....	45
CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ ĐẠT ĐƯỢC	47
4.1. Môi trường triển khai.....	47
4.2. Kiến trúc mã nguồn (Source Code)	47
4.3. Triển khai các chức năng chính.....	48
4.3.1. Module xác thực và phân quyền (Auth)	48
4.3.2. Module sản phẩm (Product/Shop)	48
4.3.3. Module giỏ hàng (Cart) và Thanh toán (Checkout)	49
4.3.4. Module quản trị (Admin Dashboard)	49
4.3.5. Module gợi ý (Recommendation)	50
4.3.6. Module xử lý ảnh và Upload file.....	50
4.3.7. Tích hợp JavaScript và Chart.js phía client.....	52
4.4. Giao diện và trải nghiệm người dùng.....	52
4.5. Đánh giá hoạt động của AI Recommender	53

4.6. Tổng kết kết quả triển khai	54
CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ	55
5.1. Chiến lược và kế hoạch kiểm thử	55
5.1.1. Mục tiêu kiểm thử	55
5.1.2. Phân loại và phạm vi kiểm thử	55
5.2.2. Kiểm thử các Model trong models.py	56
5.3.1. Kiểm thử Route Authentication.....	56
5.3.2. Kiểm thử Route Giỏ hàng.....	57
5.3.3. Kiểm thử Route Thanh toán	57
5.3.4. Kiểm thử Route Admin	57
5.4. Kiểm thử chức năng (Functional Testing — End-to-End).....	58
5.4.1. Luồng mua hàng hoàn chỉnh của Khách hàng (TC-FUNC-01)	58
5.4.2. Luồng quản trị của Admin (TC-FUNC-02)	58
5.5. Kiểm thử bảo mật (Security Testing)	58
5.5.1. Bảo vệ xác thực và phân quyền	59
5.5.2. Bảo vệ Upload và đầu vào người dùng	59
5.6. Kiểm thử hiệu năng (Performance Testing)	59
5.6.1. Thời gian phản hồi các route chính	59
5.6.2. Kiểm thử cơ chế Cache Recommender	60
5.6.3. Kiểm thử áp lực (Stress Testing).....	60
5.7. Kiểm thử khả năng sử dụng (Usability) và Tương thích (Compatibility).....	60
5.7.1. Kiểm thử khả năng sử dụng (Usability)	60
5.7.2. Kiểm thử tương thích trình duyệt (Browser Compatibility)	61
5.7.3. Kiểm thử khả năng đáp ứng (Responsive Design).....	61
5.8. Kiểm thử Hồi quy (Regression Testing) và Quản lý lỗi.....	62
5.8.1. Bộ test Hồi quy	62
5.8.2. Quản lý lỗi và bug đã phát hiện trong quá trình kiểm thử.....	62

5.9. Tổng kết kết quả kiểm thử	63
5.9.1. Bảng tổng hợp kết quả	63
5.9.2. Nhận xét tổng quan	64
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	65
6.1. Tổng kết kết quả nghiên cứu	65
TÀI LIỆU THAM KHẢO	66
PHỤ LỤC	67
Phụ lục A — Cấu trúc thư mục dự án	67
Phụ lục B — Danh sách các route/endpoint chính	68
Phụ lục C — Mã nguồn tiêu biểu của class HybridRecommender	69
Phụ lục D — Hướng dẫn cài đặt và chạy hệ thống	72
D.1. Yêu cầu hệ thống	72
D.2. Các bước cài đặt	72
D.3. Cấu trúc dữ liệu mẫu	72
D.4. Khắc phục sự cố thường gặp	73
Phụ lục E — Mã DDL khởi tạo database	74
Phụ lục F — Dữ liệu seed mẫu (sample data)	76
Phụ lục G — Bảng thuật ngữ (Glossary)	77
Phụ lục H — Tổng hợp các Prompt đã dùng trong dự án	79
H.1. Prompt giai đoạn thiết kế	79
H.2. Prompt giai đoạn lập trình	79
H.3. Prompt giai đoạn debug	80
H.4. Prompt giai đoạn kiểm thử	80
H.5. Prompt giai đoạn viết tài liệu	81

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, hoạt động mua sắm trực tuyến phát triển mạnh và đã trở thành một thói quen tiêu dùng phổ biến. Bên cạnh các nhóm sản phẩm như thời trang hay điện tử, ngành thực phẩm và đồ ăn cũng tăng dần chuyển dịch lên môi trường số. Người dùng ngày nay có xu hướng đặt mua thực phẩm thông qua các trang web, ứng dụng để tiết kiệm thời gian, tránh việc phải chuyển đến cửa hàng truyền thống, đồng thời dễ dàng so sánh nhiều mặt hàng với nhau trước khi mua.

Tuy vậy, song song với sự tiện lợi đó cũng xuất hiện một số vấn đề. Số lượng sản phẩm trên các cửa hàng trực tuyến thường rất lớn, được phân thành nhiều danh mục khác nhau như rau củ, trái cây, thịt tươi, hải sản, đồ uống, gia vị, đồ khô, bánh kẹo... Người dùng phải bỏ ra khá nhiều thời gian để tìm đúng sản phẩm phù hợp với nhu cầu, sở thích và mức giá mong muốn. Trong nhiều trường hợp, người dùng có thể bỏ qua giỏ hàng hoặc rời khỏi website chỉ vì không tìm thấy đúng món cần mua, hoặc vì hiển thị sản phẩm không liên quan đến thói quen tiêu dùng của họ.

Xuất phát từ thực tế đó, nhóm chọn đề tài “FreshMart — Web bán thực phẩm/đồ ăn tích hợp gợi ý sản phẩm bằng AI”. Hệ thống được xây dựng nhằm cung cấp đầy đủ các chức năng cốt lõi của một trang web bán hàng (xem sản phẩm, giỏ hàng, đặt hàng, theo dõi đơn), đồng thời bổ sung cơ chế gợi ý sản phẩm dựa trên hành vi và sở thích người dùng để hỗ trợ ra quyết định mua sắm. Mục đích là giúp rút ngắn thời gian tìm kiếm sản phẩm, tăng trải nghiệm mua sắm và đồng thời gia tăng khả năng người dùng quay lại sử dụng hệ thống.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan

trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

Đề tài cũng phù hợp với yêu cầu môn học là vận dụng GenAI và Vibe Engineering trong quá trình phát triển phần mềm, không chỉ dừng ở việc mô tả chức năng hệ thống mà còn thể hiện được vai trò của AI trong development pipeline.

1.2 Mục tiêu nghiên cứu

Mục tiêu thứ nhất là xây dựng được một website bán thực phẩm hoàn chỉnh ở mức cơ bản. Website cần thực hiện được luồng nghiệp vụ trọn vẹn từ khi người dùng đăng nhập, duyệt sản phẩm, thêm vào giỏ hàng, tiến hành đặt hàng (checkout) cho đến khi xem lại lịch sử đơn hàng đã đặt. Đây là phần nền tảng để chứng minh hệ thống vận hành đúng nghiệp vụ thương mại điện tử, không chỉ là một trang web tĩnh để xem sản phẩm.

Mục tiêu thứ hai là hệ thống có phân quyền và có trang quản trị dành cho admin. Admin có thể thêm, sửa, hoặc ẩn sản phẩm; xem dashboard tổng quan; và cập nhật trạng thái đơn hàng (chờ xử lý, đã xác nhận, đang giao, hoàn thành, đã hủy). Điều này đảm bảo hệ thống có thể vận hành thực tế bởi nhân sự quản trị, không phụ thuộc vào người làm kỹ thuật.

Mục tiêu thứ ba là nghiên cứu và triển khai mô hình gợi ý sản phẩm dựa trên dữ liệu. Hệ thống cần gợi ý được sản phẩm tương tự khi người dùng xem chi tiết một sản phẩm, đồng thời gợi ý cá nhân hóa dựa trên lịch sử mua hàng khi người dùng đã có dữ liệu hành vi.

Mục tiêu thứ tư là đánh giá được ưu điểm, hạn chế và điều kiện vận hành của mô hình. Báo cáo không chỉ trình bày kết quả, mà còn phân tích vì sao chọn phương pháp lai, cách xử lý Cold Start, cách tổ chức dữ liệu và cách kiểm thử module gợi ý.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan

trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

1.3 Đối tượng và phạm vi đề tài

Đối tượng nghiên cứu là hệ thống website bán thực phẩm quy mô vừa và nhỏ, trong đó sản phẩm được tổ chức theo danh mục và khách hàng đặt hàng trực tuyến. Đối tượng kỹ thuật là các thành phần web backend, frontend, cơ sở dữ liệu quan hệ và hệ thống gợi ý sản phẩm.

Phạm vi chức năng bao gồm nhóm người dùng khách hàng và nhóm quản trị viên. Khách hàng có thể duyệt và mua sản phẩm; quản trị viên có thể quản lý sản phẩm, danh mục, tồn kho và trạng thái đơn hàng. Báo cáo chưa đi sâu vào thanh toán thực tế qua ngân hàng hoặc vận chuyển thời gian thực. Hệ thống được thiết kế cho ba nhóm đối tượng người dùng với các quyền khác nhau:

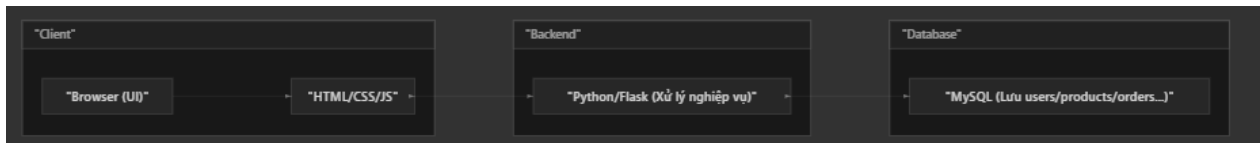
- Guest (khách chưa đăng nhập): Đây là nhóm người dùng truy cập website nhưng chưa có tài khoản hoặc chưa đăng nhập. Guest có thể duyệt danh sách sản phẩm, tìm kiếm sản phẩm theo tên, lọc theo danh mục và xem chi tiết từng sản phẩm. Nhóm này chưa thể thực hiện các thao tác liên quan đến giỏ hàng, đặt hàng hoặc lịch sử đơn vì cần xác thực người dùng.
- User (khách đã đăng nhập): Đây là nhóm người dùng cuối chính của hệ thống. Sau khi đăng nhập, người dùng có toàn bộ quyền của Guest và bổ sung các quyền: thêm sản phẩm vào giỏ hàng, cập nhật số lượng hoặc xóa sản phẩm trong giỏ, tiến hành thanh toán (checkout) và đặt hàng, xem lại lịch sử đơn hàng đã đặt. Đây là vai trò trung tâm khi mô tả các luồng nghiệp vụ của hệ thống.
- Admin (quản trị viên): Là người chịu trách nhiệm quản lý dữ liệu và vận hành cửa hàng. Admin có thể truy cập trang quản trị (dashboard), thực hiện các thao tác CRUD đối với sản phẩm (thêm/sửa/xóa), quản lý danh sách đơn hàng và cập nhật trạng thái đơn. Việc tách quyền Admin ra khỏi User giúp hệ thống có sự phân chia trách nhiệm rõ ràng, đảm bảo bảo mật cũng như đảm bảo trải nghiệm tập trung cho từng nhóm.

Phạm vi thuật toán tập trung vào Content-Based Filtering, Collaborative Filtering và mô hình Hybrid kết hợp. Các kỹ thuật sử dụng gồm TF-IDF, Cosine Similarity, User-Item Matrix và cách tính điểm tổng hợp có trọng số.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

1.4 Kiến trúc tổng quan



Hệ thống FreshMart được xây dựng theo mô hình kiến trúc 3 lớp (3-tier architecture) kinh điển — gồm tầng Trình bày (Presentation), tầng Nghiệp vụ (Business Logic) và tầng Dữ liệu (Data). Đây là mô hình kiến trúc rất phổ biến trong các ứng dụng web vì có sự phân tách trách nhiệm rõ ràng giữa các phần, giúp dễ phát triển song song, dễ bảo trì và dễ mở rộng. Trong phạm vi đồ án, mô hình này phù hợp vì hệ thống không quá lớn, không yêu cầu phân tán hoặc xử lý hiệu năng cao, đồng thời triển khai được trên một máy duy nhất phục vụ cho việc chạy demo và đánh giá.

Tầng Trình bày (Presentation Layer — Browser/Client): Đây là phần mà người dùng nhìn thấy và thao tác trực tiếp. Tầng này được hiển thị trên trình duyệt web và sử dụng các công nghệ web nền tảng: HTML để cấu trúc trang, CSS để định dạng giao diện, và JavaScript để bổ sung tương tác phía client (tăng/giảm số lượng trong giỏ, hiển thị thông báo, validate form đơn giản...). Tầng trình bày không chứa logic nghiệp vụ; mọi thao tác cần dữ liệu hoặc tính toán đều được gửi về server xử lý. Nhờ vậy, phần giao diện luôn “gọn nhẹ”, dễ chỉnh sửa và phản hồi nhanh.

Tầng Nghiệp vụ (Business Logic Layer — Backend): Đây là phần “bộ não” của hệ thống, được viết bằng Python với framework Flask. Tầng này nhận request từ client, thực hiện toàn bộ logic nghiệp vụ và trả về kết quả.

Tầng Dữ liệu (Data Layer — MySQL): Đây là nơi lưu trữ toàn bộ dữ liệu lâu dài của hệ thống: tài khoản người dùng, danh mục, sản phẩm, đơn hàng, chi tiết đơn hàng. Tầng này được giao tiếp thông qua SQLAlchemy ORM thay vì viết SQL trực tiếp, giúp code rõ ràng hơn và giảm nguy cơ SQL injection. Việc kết nối tới MySQL được thực hiện qua driver PyMySQL. Hệ thống cũng có cơ chế tự động kiểm tra và tạo database nếu chưa tồn tại, giúp việc triển khai và demo dễ dàng hơn.

1.5 Ý nghĩa thực tiễn

Một website bán thực phẩm có gợi ý sản phẩm giúp khách hàng rút ngắn thời gian mua sắm, phát hiện thêm sản phẩm phù hợp và hạn chế việc bỏ sót sản phẩm thường mua. Với doanh nghiệp, hệ thống gợi ý có thể tăng doanh thu thông qua bán chéo, bán thêm và nâng cao khả năng giữ chân khách hàng.

Về mặt học thuật, đề tài là ví dụ thực tế cho việc kết hợp phát triển phần mềm với khai phá dữ liệu. Sinh viên có cơ hội hiểu rõ chu trình từ phân tích yêu cầu, thiết kế cơ sở dữ liệu, lập trình chức năng đến ứng dụng thuật toán vào sản phẩm phần mềm.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về thương mại điện tử thực phẩm

Thương mại điện tử thực phẩm có đặc thù khác với nhiều ngành hàng khác. Sản phẩm thường có hạn sử dụng, yêu cầu thông tin về đơn vị tính, tồn kho, nguồn gốc và trạng thái còn hàng. Người dùng thường mua theo nhóm sản phẩm liên quan, ví dụ mua thịt đi kèm gia vị, mua rau củ đi kèm thực phẩm tươi sống hoặc mua trái cây theo mùa.

Do đặc thù mua lặp lại, website thực phẩm rất phù hợp để áp dụng cá nhân hóa. Nếu hệ thống ghi nhận được lịch sử đơn hàng, tần suất mua và nhóm sản phẩm thường đi cùng nhau, module gợi ý có thể đưa ra các đề xuất hữu ích hơn so với việc chỉ hiển thị sản phẩm bán chạy.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

2.2 Hệ thống gợi ý

Hệ thống gợi ý là thành phần phần mềm có nhiệm vụ lựa chọn và sắp xếp các item phù hợp với từng người dùng. Trong đồ án này, item là sản phẩm thực phẩm, user là khách hàng, còn tương tác chính được sử dụng là hành vi mua hàng hoặc số lượng sản phẩm trong đơn.

Có nhiều nhóm kỹ thuật gợi ý khác nhau. Content-Based Filtering dựa vào đặc trưng nội dung của sản phẩm. Collaborative Filtering dựa vào hành vi của cộng đồng người dùng. Hybrid Recommendation kết hợp nhiều nguồn tín hiệu để tạo kết quả ổn định hơn. Đề tài chọn mô hình lai vì dữ liệu thương mại điện tử thực phẩm có cả thông tin mô tả sản phẩm và lịch sử mua hàng.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

2.3 Content-Based Filtering

Content-Based Filtering giả định rằng nếu người dùng quan tâm một sản phẩm, họ có khả năng quan tâm những sản phẩm có nội dung tương tự. Nội dung có thể là tên sản phẩm, mô tả, danh mục, từ khóa, giá, đơn vị tính hoặc nhãn sản phẩm. Với dữ liệu văn bản, phương pháp phổ biến là chuyển văn bản thành vector số để tính độ tương tự.

TF-IDF được dùng để biểu diễn mức độ quan trọng của từ trong một sản phẩm so với toàn bộ tập sản phẩm. Từ xuất hiện nhiều trong một mô tả nhưng ít xuất hiện ở toàn bộ kho sản phẩm sẽ có trọng số cao hơn. Sau khi có vector TF-IDF, Cosine Similarity được dùng để đo góc giữa hai vector. Giá trị càng gần 1 thì hai sản phẩm càng giống nhau.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

2.4 Collaborative Filtering

Collaborative Filtering dựa vào giả định rằng những người dùng có hành vi giống nhau trong quá khứ có thể tiếp tục có sở thích tương tự trong tương lai. Trong đồ án, hành vi được biểu diễn qua ma trận User-Item, trong đó hàng là người dùng, cột là sản phẩm và giá trị là số lượng đã mua.

Với User-based Collaborative Filtering, hệ thống tính độ tương tự giữa các người dùng, tìm nhóm người dùng gần nhất với người dùng hiện tại, sau đó đề xuất các sản phẩm mà nhóm tương tự đã mua nhưng người dùng hiện tại chưa mua. Cách tiếp cận này giúp phát hiện sản phẩm bổ sung mà mô tả nội dung chưa chắc thể hiện rõ.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

2.5 Hybrid Recommendation

Mô hình lai được dùng để khắc phục hạn chế của từng phương pháp đơn lẻ. Content-Based Filtering hoạt động tốt với sản phẩm mới vì chỉ cần có mô tả, nhưng dễ lặp lại nhóm sản phẩm tương tự. Collaborative Filtering có thể gợi ý đa dạng hơn, nhưng cần dữ liệu hành vi đủ lớn và gặp khó khăn với người dùng mới.

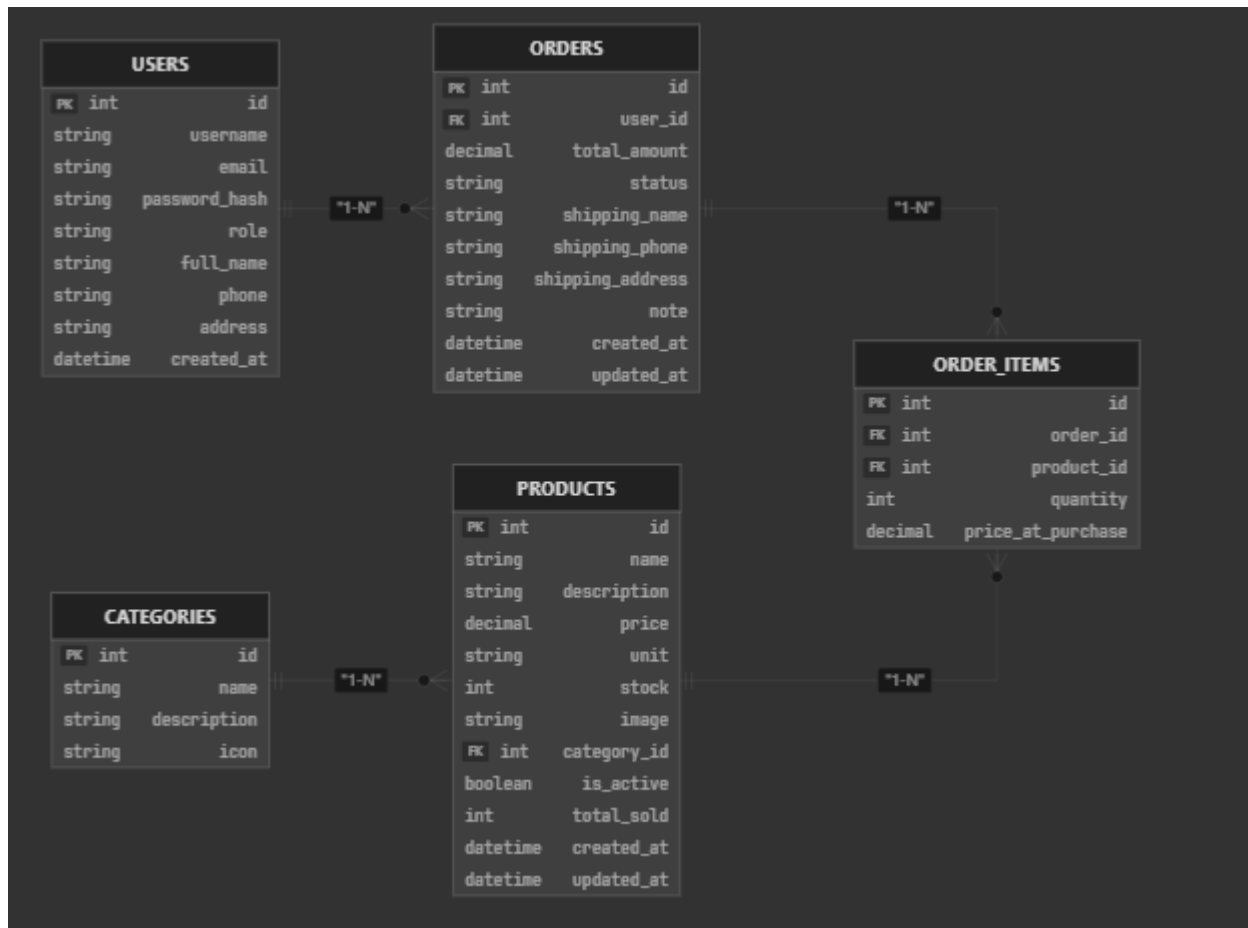
Trong hệ thống đề xuất, điểm cuối cùng được tính bằng công thức: $\text{Final_Score} = \alpha * \text{CF_Score} + (1 - \alpha) * \text{CBF_Score}$. Khi người dùng có lịch sử mua hàng, α có thể đặt ở mức cân bằng. Khi người dùng mới chưa có lịch sử, hệ thống giảm trọng số CF và ưu tiên CBF hoặc sản phẩm bán chạy.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình

mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

2.6 Mô hình dữ liệu (ERD)



Hệ thống FreshMart sử dụng cơ sở dữ liệu quan hệ MySQL với năm bảng chính, mỗi bảng phục vụ một phần nghiệp vụ rõ ràng và được thiết kế để có thể mở rộng thêm trong các giai đoạn phát triển sau. Dưới đây là mô tả chi tiết từng bảng.

Bảng users lưu thông tin tài khoản và hồ sơ của người dùng hệ thống. Mỗi bản ghi đại diện cho một tài khoản, được định danh duy nhất bởi trường id đóng vai trò khóa chính. Hai trường username và email được khai báo UNIQUE để đảm bảo không có hai tài khoản trùng tên đăng nhập hoặc trùng email; trường password_hash lưu mật khẩu dưới

dạng hash bằng `werkzeug.security.generate_password_hash` thay vì lưu mật khẩu thô, qua đó đảm bảo an toàn ngay cả khi dữ liệu bị truy cập trái phép. Trường `role` đóng vai trò phân quyền với hai giá trị chính là `user` và `admin`, là cơ sở để hệ thống điều hướng đúng giao diện sau khi đăng nhập (ví dụ `admin` sẽ vào dashboard quản trị, `user` thường sẽ vào trang chủ shop). Các trường `full_name`, `phone`, `address` lưu thông tin cá nhân phục vụ cho việc giao hàng và liên lạc; còn `created_at` ghi nhận thời điểm tạo tài khoản, hỗ trợ cho việc thống kê người dùng mới và phân tích hành vi theo thời gian.

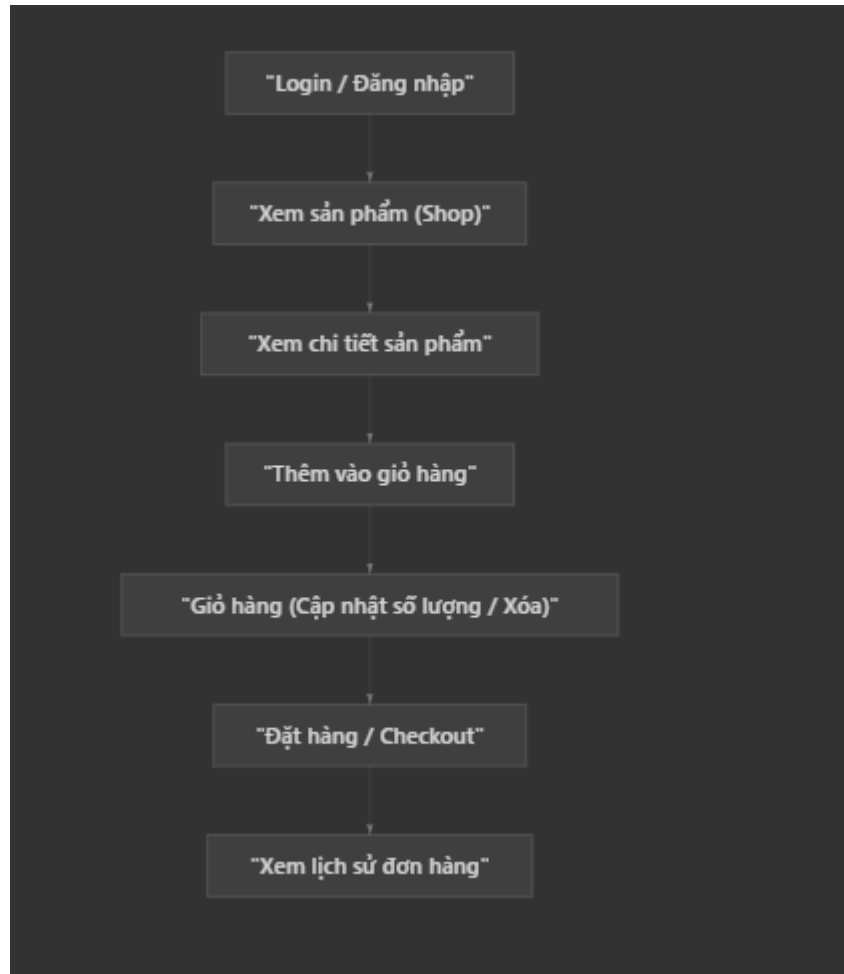
Bảng `categories` lưu trữ các danh mục thực phẩm mà cửa hàng kinh doanh, ví dụ như Rau củ quả, Trái cây, Thịt tươi, Hải sản, Đồ uống, Gia vị, Đồ khô & Mì, Bánh kẹo... Mỗi danh mục được định danh bởi trường `id` (khóa chính) và có một trường `name` được khai báo `UNIQUE` để tránh tạo trùng tên danh mục. Bên cạnh đó, bảng còn có `description` để mô tả ngắn về danh mục (giúp hiển thị trên trang shop và phục vụ cho phần gợi ý dựa trên nội dung), và `icon` để lưu tên icon đại diện hiển thị trên giao diện. Bảng `categories` tuy đơn giản nhưng đóng vai trò quan trọng trong việc tổ chức sản phẩm theo nhóm, hỗ trợ chức năng lọc theo danh mục ở trang Shop, và là một trong các tín hiệu chính cho thuật toán Content-based Recommendation.

Bảng `products` là bảng trung tâm của hệ thống, lưu thông tin chi tiết của từng sản phẩm thực phẩm được bán. Mỗi sản phẩm được định danh bằng trường `id` (khóa chính) và có hai trường mô tả là `name` (tên sản phẩm) và `description` (mô tả chi tiết). Trường `price` lưu giá bán hiện tại của sản phẩm và được sử dụng kiểu `Decimal` thay vì `Float` nhằm đảm bảo độ chính xác cao trong tính toán tiền tệ — tránh các lỗi sai số do biểu diễn dấu phẩy động. Trường `unit` lưu đơn vị tính (như `kg`, `hộp`, `chai`, `gói`) phục vụ cho việc hiển thị giá theo đúng đơn vị bán; `stock` ghi nhận số lượng tồn kho hiện có và được tự động trừ đi khi có đơn hàng được tạo thành công. Trường `image` lưu tên file ảnh đại diện cho sản phẩm (lưu trong thư mục `static/uploads/`), với giá trị mặc định là `default_food.svg` để tránh lỗi hiển thị khi chưa có ảnh thật. Trường `category_id` là khóa ngoại liên kết tới bảng `categories`, biểu thị quan hệ “một sản phẩm thuộc về một danh mục”. Đáng chú ý, bảng có trường `is_active` đóng vai trò “soft delete” — khi `admin` muốn ngừng bán một sản phẩm, hệ thống chỉ đặt `is_active = false` thay vì xóa cứng, giúp giữ nguyên lịch sử đơn hàng đã có sản phẩm đó. Trường `total_sold` đếm tổng số lượt sản phẩm đã được bán ra, phục vụ cho việc thống kê doanh số và làm cơ sở cho tính năng “bán chạy” (bestseller). Cuối cùng, các trường `created_at` và `updated_at` ghi nhận thời điểm tạo và lần chỉnh sửa gần nhất của sản phẩm, hỗ trợ cho tiêu chí sắp xếp “mới nhất” trên trang Shop.

Bảng `orders` lưu thông tin các đơn hàng được tạo bởi người dùng. Mỗi lần người dùng hoàn tất quy trình checkout, một bản ghi mới sẽ được sinh trong bảng này, được định danh bằng trường `id` (khóa chính). Trường `user_id` là khóa ngoại liên kết tới bảng `users` để xác định đơn hàng này thuộc về ai — cũng là cơ sở cho chức năng “Lịch sử đơn hàng” khi người dùng quay lại tra cứu. Trường `total_amount` (kiểu `Decimal`) lưu tổng tiền đơn hàng tại thời điểm đặt, được tính bằng cách cộng dồn $\text{quantity} \times \text{price_at_purchase}$ của tất cả các dòng chi tiết đơn hàng. Trường `status` thể hiện trạng thái xử lý đơn hàng và nhận một trong năm giá trị: `pending` (chờ xử lý — trạng thái mặc định khi đơn vừa được tạo), `confirmed` (đã được admin xác nhận), `shipping` (đang trong quá trình giao hàng), `completed` (đã giao thành công và hoàn tất), và `cancelled` (đã hủy). Trường `status` cho phép admin theo dõi và cập nhật tiến trình đơn hàng, đồng thời để người dùng tự xem trạng thái của các đơn họ đã đặt. Các trường `shipping_name`, `shipping_phone`, `shipping_address` và `note` lưu thông tin giao hàng do người dùng nhập tại bước checkout — được thiết kế tách biệt với hồ sơ trong bảng `users` để đảm bảo: nếu người dùng đặt hàng cho người khác hoặc tạm thời gửi đến địa chỉ khác, thông tin giao hàng vẫn được lưu chính xác cho từng đơn. Hai trường `created_at` và `updated_at` ghi nhận thời điểm tạo đơn và lần cập nhật trạng thái gần nhất, phục vụ cho việc thống kê và sắp xếp.

Bảng `order_items` lưu chi tiết từng dòng sản phẩm trong một đơn hàng — mỗi dòng tương ứng với một loại sản phẩm và số lượng cụ thể trong đơn. Bảng có khóa chính là `id` và hai khóa ngoại quan trọng: `order_id` liên kết tới bảng `orders` (cho biết dòng này thuộc đơn hàng nào) và `product_id` liên kết tới bảng `products` (cho biết là sản phẩm nào). Trường `quantity` ghi số lượng sản phẩm khách đặt trong đơn, còn trường `price_at_purchase` đóng vai trò đặc biệt quan trọng: đây là một “snapshot” lưu giá tại thời điểm khách mua. Việc lưu giá riêng theo từng dòng đơn — thay vì luôn tham chiếu vào trường `price` của bảng `products` — đảm bảo rằng nếu sau này admin thay đổi giá sản phẩm, các đơn hàng cũ vẫn giữ nguyên được giá trị tại thời điểm đặt, không bị thay đổi “ngược lịch sử”. Bảng `order_items` về bản chất chính là bảng trung gian thể hiện quan hệ N–N giữa `orders` và `products`, và là nơi chứa toàn bộ dữ liệu hành vi mua hàng của người dùng. Đây cũng chính là nguồn dữ liệu đầu vào quan trọng nhất cho mô-đun Recommendation: thông qua việc thống kê các sản phẩm đã được mua bởi nhiều người dùng (cùng nhau hoặc tương tự nhau), hệ thống có thể xây dựng ma trận user-item phục vụ cho thuật toán Collaborative Filtering ở các giai đoạn phát triển sau.

2.7 Luồng nghiệp vụ chính của người dùng



Luồng nghiệp vụ chính trong hệ thống xoay quanh người dùng cuối (User). Theo thứ tự, một người dùng sẽ trải qua các bước sau khi sử dụng hệ thống:

- Đăng nhập (Login): Người dùng nhập username và mật khẩu, hệ thống xác thực bằng `check_password_hash`. Nếu đúng, người dùng được đăng nhập vào hệ thống.
- Xem sản phẩm: Sau khi đăng nhập, người dùng truy cập trang Shop, tìm kiếm theo tên, lọc theo danh mục hoặc sắp xếp theo các tiêu chí (mới nhất, giá tăng/giảm, bán chạy). Người dùng có thể xem chi tiết một sản phẩm bất kỳ.
- Thêm vào giỏ hàng: Khi quyết định mua, người dùng nhấn nút “Thêm vào giỏ”. Hệ thống cập nhật giỏ hàng (lưu trong session). Người dùng có thể vào trang Giỏ hàng để cập nhật số lượng hoặc xóa sản phẩm.
- Đặt hàng (Checkout): Người dùng chuyển sang trang Checkout, nhập thông tin giao hàng (họ tên, số điện thoại, địa chỉ, ghi chú nếu có) rồi xác nhận đặt

hàng. Hệ thống sẽ tạo bản ghi mới trong bảng orders, đồng thời tạo các dòng order_items tương ứng và cập nhật tồn kho cho từng sản phẩm.

- Xem lịch sử đơn hàng: Người dùng có thể vào trang Lịch sử đơn hàng để theo dõi các đơn đã đặt và trạng thái hiện tại của từng đơn.

2.8 Công nghệ sử dụng

Flask được chọn vì nhẹ, linh hoạt và phù hợp với đồ án web quy mô vừa. SQLAlchemy hỗ trợ ánh xạ đối tượng - quan hệ, giúp định nghĩa model rõ ràng và thao tác cơ sở dữ liệu thuận tiện. Flask-Login quản lý phiên đăng nhập, phân biệt người dùng thường và quản trị viên.

Scikit-learn cung cấp TfidfVectorizer và cosine_similarity, giúp triển khai module gợi ý nhanh và đáng tin cậy. Pandas hỗ trợ xử lý bảng dữ liệu, tạo pivot table cho ma trận User-Item. Bootstrap, HTML, CSS và Jinja2 được sử dụng để xây dựng giao diện.

MySQL là hệ quản trị cơ sở dữ liệu quan hệ phổ biến, ổn định và phù hợp với mô hình dữ liệu của bài toán bán hàng. Hệ thống cần lưu trữ nhiều bảng có quan hệ chặt chẽ với nhau (User, Category, Product, Order, OrderItem), nên MySQL là lựa chọn hợp lý. Charset được sử dụng là utf8mb4 để hỗ trợ tốt tiếng Việt.

Về mặt triển khai, nội dung này cần được xem xét đồng thời ở ba góc độ: nghiệp vụ, dữ liệu và trải nghiệm người dùng. Nếu chỉ tập trung vào giao diện mà bỏ qua dữ liệu lịch sử, hệ thống sẽ khó cá nhân hóa. Ngược lại, nếu chỉ chú trọng thuật toán mà quy trình mua hàng không thuận tiện, người dùng sẽ không tạo ra đủ tương tác để hệ thống học được sở thích.

Trong phạm vi đồ án, giải pháp được lựa chọn theo hướng cân bằng giữa tính khả thi và tính học thuật. Các thành phần được thiết kế đủ rõ để chứng minh quy trình vận hành, đồng thời vẫn giữ cấu trúc mở để có thể nâng cấp trong tương lai. Đây là điểm quan trọng vì một hệ thống thương mại điện tử thực tế luôn thay đổi theo nhu cầu kinh doanh, số lượng sản phẩm và hành vi khách hàng.

2.9 Vibe Engineering và GenAI trong phát triển phần mềm

2.9.1 Workflow dùng AI trong development pipeline

Nhóm áp dụng quy trình lặp gồm 6 bước:

Bước 1 — Nhập đầu vào: Nhóm chuẩn bị thông tin đầu vào gồm user story, phạm vi tính năng cần làm, các ràng buộc kỹ thuật (Flask, MySQL, charset utf8mb4), và tiêu chí nghiệm thu cụ thể.

Bước 2 — Prompt cho AI: Nhóm soạn prompt rõ ràng, có chỉ định dạng đầu ra mong muốn (ví dụ: ERD theo Mermaid erDiagram, Flow theo Mermaid flowchart, hay code Python theo cấu trúc Flask).

Bước 3 — AI sinh bản nháp: AI trả về bản nháp ban đầu (sơ đồ ERD, sơ đồ luồng, code khung, schema database...). Đây là phần “gợi ý ban đầu” chứ chưa phải sản phẩm cuối cùng.

Bước 4 — Review và chỉnh sửa: Nhóm đọc lại kết quả, đối chiếu với nghiệp vụ thực tế và yêu cầu, từ đó bổ sung trường còn thiếu, sửa logic chưa đúng, hoặc làm rõ phần AI chưa hiểu.

Bước 5 — Chạy thử và debug: Phần code/sơ đồ sau khi chỉnh sẽ được tích hợp vào dự án, chạy thử và quan sát log lỗi nếu có. Nếu phát sinh vấn đề, tiếp tục lặp lại bước 2 với prompt cụ thể hơn.

Bước 6 — Chốt phiên bản: Sau khi chạy ổn, kết quả được chốt và đưa vào tài liệu (slide/báo cáo) hoặc commit vào codebase.

Sơ đồ dưới đây minh họa quy trình 6 bước nói trên dưới dạng flowchart:



2.9.2 Vai trò của AI trong development pipeline

Trong dự án FreshMart, AI được tích hợp xuyên suốt nhiều giai đoạn phát triển phần mềm chứ không chỉ dừng ở một bước riêng lẻ, đóng vai trò như một “thành viên ảo” luôn sẵn sàng sinh bản nháp và phản hồi nhanh các yêu cầu của nhóm.

Ở khâu thiết kế tài liệu và phân tích yêu cầu, AI hỗ trợ phác thảo dàn ý báo cáo, outline slide theo các mốc nộp, cũng như đề xuất schema cơ sở dữ liệu, sơ đồ ERD và sơ đồ luồng nghiệp vụ dạng Mermaid khi nhóm cung cấp user story và phạm vi. Đáng chú ý, AI đã chủ động gợi ý một số trường nhóm chưa nghĩ tới, ví dụ `price_at_purchase` để giữ snapshot giá tại thời điểm mua hoặc `is_active` để hỗ trợ soft-delete sản phẩm.

Ở khâu sinh code khung (skeleton) và dữ liệu mẫu, AI giúp tạo nhanh các route Flask, template HTML, model SQLAlchemy cho các chức năng chính (Login, Shop, Cart, Checkout, Order History, Admin), cũng như sinh seed data cho 8 danh mục thực phẩm và các sản phẩm với giá/đơn vị hợp lý. Bản nháp AI sinh ra thường chạy được khoảng 60–80%, phần còn lại nhóm tinh chỉnh để khớp với nghiệp vụ và đồng bộ UI.

Ở khâu refactor và debug, AI đóng vai trò một “code reviewer” thâm lặng — gợi ý tách module, viết gọn các đoạn if/else lồng nhau bằng helper function, và đặc biệt hữu ích khi xử lý lỗi: chỉ cần copy log lỗi (ví dụ `OperationalError 1045/1049, 404 ảnh, lỗi Jinja2...`) là AI có thể nhanh chóng chỉ ra nguyên nhân và đề xuất hướng fix, giúp nhóm khoan vùng vấn đề nhanh hơn so với tự tra cứu thủ công.

Tóm lại, AI tham gia rộng khắp trong development pipeline — từ tài liệu, thiết kế, code, dữ liệu cho đến refactor và debug — nhưng vai trò của AI luôn dừng ở mức “đề xuất bản nháp”. Nhóm phát triển vẫn là người đánh giá, chỉnh sửa và chịu trách nhiệm về chất lượng cuối cùng của sản phẩm, đúng với tinh thần Vibe Engineering: tận dụng tối đa năng lực sinh nhanh của AI nhưng vẫn giữ vững quyền kiểm soát của con người.

2.8.3 Minh họa Prompt → Code sinh ra → Chỉnh sửa

Minh họa 1 — Tạo trang Shop với lọc/tìm kiếm/sắp xếp

- Prompt: “Tạo trang /shop cho web bán thực phẩm bằng Flask + MySQL: hiển thị danh sách sản phẩm, có tìm kiếm theo tên, lọc theo danh mục, sắp xếp theo các tiêu chí (mới nhất, giá tăng, giá giảm, bán chạy) và có phân trang.”
- AI sinh ra: Route /shop với các tham số q, category, sort, page; phần query SQLAlchemy tương ứng; template shop.html với sidebar bộ lọc và lưới sản phẩm.
- Nhóm chỉnh sửa: Tinh chỉnh URL parameter để gọn hơn, cập nhật phần xử lý khi không có kết quả tìm kiếm (hiển thị thông báo phù hợp), và đồng bộ phong cách UI với phần còn lại của hệ thống.

Minh họa 2 — Tạo chức năng Login

- Prompt: “Tạo trang Login cho web bán thực phẩm bằng Flask + MySQL: form username/password, xác thực bằng password hash, login thành công thì redirect về trang chủ (hoặc admin dashboard nếu là admin), login sai thì báo lỗi.”
- AI sinh ra: Route /login xử lý GET/POST, template login.html, logic kiểm tra user và mật khẩu bằng `check_password_hash`.
- Nhóm chỉnh sửa: Bổ sung validate input, xử lý tham số next để quay lại trang trước khi đăng nhập, thống nhất thông báo lỗi/thành công theo UI chung của hệ thống.

Minh họa 3 — Tự tạo database nếu chưa tồn tại

- Prompt: “Tự tạo database food_store_db nếu chưa tồn tại trước khi chạy app/seed bằng SQLAlchemy + PyMySQL.”
- Lỗi gặp phải trước khi fix: OperationalError (1049): "Unknown database 'food_store_db'" khi chạy seed_data.py lần đầu.
- AI sinh ra: Hàm ensure_database_exists() trong app.py, gọi trước db.create_all(), sử dụng CREATE DATABASE IF NOT EXISTS.
- Nhóm chỉnh sửa: Đổi cách kết nối — thay vì kết nối tới chính food_store_db (sẽ thất bại), kết nối tới schema mặc định mysql luôn tồn tại trên server, đặt isolation_level="AUTOCOMMIT" để DDL chạy được. Sau khi áp dụng, lệnh python seed_data.py chạy được trực tiếp mà không cần tạo tay database trước.

2.8.4 Đánh giá hiệu quả sử dụng AI

Sau quá trình áp dụng AI vào dự án FreshMart, nhóm nhận thấy AI mang lại hiệu quả rõ nhất ở các công việc có tính lặp lại hoặc cần tạo bản nháp nhanh, như phác thảo dàn ý báo cáo và slide, dựng ERD/flow bằng Mermaid, sinh code khung cho login/shop/cart/checkout/admin, và đặc biệt là hỗ trợ debug khi gặp các lỗi phổ biến của Flask/MySQL. Nhờ vậy, nhóm có phiên bản ban đầu khá nhanh để chạy thử và chỉnh sửa, thay vì mất nhiều thời gian ở các bước kỹ thuật cơ bản.

Tuy nhiên, kết quả AI sinh ra không phải lúc nào cũng khớp 100% với nghiệp vụ và môi trường chạy thực tế của nhóm — một số đoạn code phải chỉnh để phù hợp cấu hình Flask/MySQL, một số sơ đồ Mermaid cần sửa lại tên bảng và quan hệ. Vì vậy nhóm luôn đọc lại, chạy thử và chỉnh sửa trước khi sử dụng. Có thể kết luận: AI là công cụ hỗ trợ hiệu quả ở các khâu lên ý tưởng, dựng khung, refactor và debug, nhưng quyết định cuối và trách nhiệm về chất lượng sản phẩm vẫn do con người/nhóm phát triển đảm nhiệm — đúng tinh thần Vibe Engineering.

CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Tổng quan phương pháp đề xuất

Sau khi đã trình bày bối cảnh, mục tiêu, phạm vi và cơ sở lý thuyết ở Chương 1 và Chương 2, Chương 3 sẽ trình bày phương pháp đề xuất để giải quyết bài toán đặt ra. Phương pháp đề xuất ở đây không chỉ là việc “chọn công nghệ” mà còn bao gồm cách tiếp cận tổng thể: thiết kế cơ sở dữ liệu chi tiết, kiến trúc của hệ gợi ý lai (Hybrid Recommendation), cách triển khai kỹ thuật về mặt mã nguồn, và quy trình áp dụng GenAI vào quá trình phát triển.

Phương pháp đề xuất tổng thể của nhóm dựa trên ba định hướng chính. **Thứ nhất**, sử dụng kiến trúc web 3 lớp (Client – Backend – Database) với Flask + MySQL làm nền tảng vì phù hợp với quy mô đồ án và dễ triển khai. **Thứ hai**, đối với chức năng cốt lõi là gợi ý sản phẩm, nhóm đề xuất phương pháp Hybrid Recommendation — kết hợp Content-based Filtering và Collaborative Filtering — kèm chiến lược xử lý cold-start để đảm bảo hệ thống luôn có gợi ý phù hợp trong mọi tình huống. **Thứ ba**, đối với quy trình phát triển, nhóm áp dụng Vibe Engineering có hệ thống — AI sinh bản nháp, nhóm review/chỉnh sửa, chạy thử và lặp lại — để tăng tốc các bước thiết kế, dựng khung và debug, đồng thời vẫn đảm bảo chất lượng cuối cùng.

Định hướng tổng thể của nhóm dựa trên ba trụ cột chính. Trụ cột thứ nhất là kiến trúc web 3 lớp (Client – Backend – Database) với Flask + MySQL/SQLite làm nền tảng, kết hợp Flask-SQLAlchemy làm ORM và Flask-Login cho quản lý phiên đăng nhập. Trụ cột thứ hai là module Recommendation lai (Hybrid) — kết hợp Content-Based Filtering dùng TF-IDF + Cosine Similarity và Collaborative Filtering dùng User-User similarity, có cơ chế xử lý Cold-Start để hệ thống luôn có gợi ý phù hợp trong mọi tình huống. Trụ cột thứ ba là quy trình phát triển có hỗ trợ AI (Vibe Engineering) — AI sinh bản nháp, nhóm review và chỉnh sửa, chạy thử rồi lặp lại — được áp dụng xuyên suốt từ thiết kế đến triển khai.

Các phần tiếp theo của chương sẽ trình bày chi tiết từng định hướng nói trên, bắt đầu từ việc phân tích yêu cầu chi tiết để có cái nhìn đầy đủ về các chức năng cần xây dựng.

3.2 Thiết kế cơ sở dữ liệu (RDBMS qua SQLAlchemy)

Hệ thống FreshMart có ba tác nhân chính tương tác trực tiếp với hệ thống. Tác nhân thứ nhất là **Guest** — đại diện cho khách hàng truy cập website nhưng chưa đăng nhập; nhóm này có quyền duyệt sản phẩm, tìm kiếm và lọc theo danh mục nhưng chưa thể đặt hàng. Tác nhân thứ hai là **User** — khách hàng đã đăng ký và đăng nhập; tác nhân này có toàn bộ quyền của Guest cộng thêm các quyền thao tác giỏ hàng, đặt hàng, xem lịch sử đơn và nhận gợi ý cá nhân hóa. Tác nhân thứ ba là **Admin** — quản trị viên của hệ thống; chịu trách nhiệm quản lý sản phẩm, danh mục, đơn hàng và xem các báo cáo thống kê. Việc phân định rõ ba tác nhân giúp nhóm thiết kế các use case và phân quyền một cách nhất quán.

Hệ thống sử dụng mô hình cơ sở dữ liệu quan hệ (RDBMS) và được định nghĩa thông qua Flask-SQLAlchemy. Các model chính của hệ thống gồm năm bảng đã được giới thiệu ở Chương 2 (users, categories, products, orders, order_items), trong đó mỗi bảng đảm nhiệm một phần nghiệp vụ rõ ràng.

Bảng User quản lý tài khoản người dùng với mật khẩu được băm (password_hash) thông qua Werkzeug, kèm theo phân quyền role nhận hai giá trị user hoặc admin; ngoài ra còn có các trường thông tin liên hệ như họ tên, số điện thoại, địa chỉ phục vụ cho việc giao hàng. Bảng Category lưu các danh mục phân loại sản phẩm (như rau củ, thịt cá, gia vị, đồ uống...) với các trường name, description và icon đại diện. Bảng Product là bảng quản lý tập sản phẩm (Item) với các thuộc tính name, description, price, stock, unit, image, khóa ngoại tới Category, và đáng chú ý là trường total_sold đếm số lượng đã bán — đây là dữ liệu quan trọng để xếp hạng sản phẩm bán chạy phục vụ cho cơ chế Fallback của hệ gợi ý.

Bảng Order lưu thông tin đơn đặt hàng với các trạng thái pending, confirmed, shipping, completed, cancelled, và bảng OrderItem lưu chi tiết từng dòng sản phẩm trong đơn (bao gồm product_id, quantity, price_at_purchase để snapshot giá tại thời điểm mua). Hai bảng Order và OrderItem đặc biệt quan trọng vì chính chúng là nguồn dữ liệu thô (Raw Data) để xây dựng tập huấn luyện hành vi cho hệ gợi ý — cụ thể là tạo ma trận User-Item Matrix dùng trong Collaborative Filtering. Mỗi cặp (user_id, product_id, quantity) trong bảng OrderItem được xem như một “tín hiệu hành vi” thể hiện sở thích của người dùng đối với sản phẩm tương ứng.

Việc tách bảng Order và OrderItem thành hai bảng riêng biệt tuân theo nguyên tắc chuẩn hóa cơ sở dữ liệu (database normalization). Bảng Order chứa thông tin chung của một đơn hàng (người đặt, tổng tiền, trạng thái, địa chỉ giao), trong khi bảng OrderItem chứa chi tiết từng sản phẩm trong đơn đó. Cách tách này giúp hệ thống dễ dàng thêm/sửa/xóa từng dòng sản phẩm trong đơn, đồng thời cho phép thực hiện các truy vấn phân tích trên dữ liệu hành vi mà không phải parse JSON hoặc chuỗi.

3.2.1. Mô tả chi tiết từng bảng và ý nghĩa các trường

Phần này trình bày chi tiết schema của từng bảng — kèm theo kiểu dữ liệu, giá trị mặc định, ràng buộc và ý nghĩa nghiệp vụ — để phục vụ cho việc triển khai chính xác và bảo trì lâu dài. Tất cả các bảng đều có khóa chính tự tăng id INT AUTO_INCREMENT PRIMARY KEY và hai trường thời gian created_at, updated_at để hỗ trợ truy vết, kiểm toán dữ liệu (audit trail) khi cần điều tra sự cố.

Bảng users lưu thông tin tài khoản người dùng và là nguồn dữ liệu cho cả module Authentication lẫn cho việc tính ma trận User-Similarity trong Collaborative Filtering. Các trường chính bao gồm username VARCHAR(64) UNIQUE NOT NULL (định danh đăng nhập, không cho phép trùng), email VARCHAR(120) UNIQUE NOT NULL (định danh thay thế, sử dụng để khôi phục mật khẩu trong tương lai), password_hash VARCHAR(255) NOT NULL (chuỗi băm theo chuẩn pbkdf2:sha256 của Werkzeug — chiều dài lớn để có thể chứa cả salt lẫn nhiều vòng băm), role ENUM('user','admin') DEFAULT 'user' (phân quyền hai cấp), full_name VARCHAR(120) (họ tên hiển thị), phone VARCHAR(20) (số điện thoại liên hệ), address TEXT (địa chỉ mặc định để giao hàng). Trường is_active TINYINT(1) DEFAULT 1 được dùng để vô hiệu hóa tài khoản (soft delete) thay vì xóa hẳn — vừa giữ được lịch sử mua hàng vừa tuân thủ yêu cầu về dữ liệu của các nghiệp vụ lâu dài.

Bảng categories chứa các danh mục phân loại sản phẩm. Mỗi danh mục có name VARCHAR(80) UNIQUE NOT NULL, description VARCHAR(255) (mô tả ngắn để hiển thị trên trang shop), icon VARCHAR(80) (tên icon hoặc đường dẫn ảnh đại diện danh mục, ví dụ "icon-rau-cu.svg"). Trường này không có nhiều cột vì đặc thù của FreshMart là số danh mục ít (~8 danh mục) và mỗi danh mục mang ý nghĩa phân loại thuần túy. Tuy nhiên, category_name lại đóng vai trò rất quan trọng trong Content-Based Filtering vì chính nó được lặp hai lần trong chuỗi content để tăng trọng số.

Bảng products là bảng trung tâm và phức tạp nhất, lưu tất cả thông tin sản phẩm. Các trường gồm: name VARCHAR(200) NOT NULL, description TEXT (mô tả dài, dùng cho cả hiển thị và TF-IDF), price DECIMAL(12,2) NOT NULL (giá bán hiện tại, dùng DECIMAL thay vì FLOAT để tránh sai số phép tính tiền tệ), stock INT DEFAULT 0 (số lượng tồn kho hiện tại — sẽ giảm khi đơn hàng được tạo và tăng khi đơn hàng bị hủy), unit VARCHAR(20) (đơn vị tính như "kg", "gói", "chai", "túi"), image

VARCHAR(255) DEFAULT 'default_food.svg' (tên file ảnh trong thư mục static/uploads/), category_id INT NOT NULL FOREIGN KEY REFERENCES categories(id), total_sold INT DEFAULT 0 (số lượng đã bán tích lũy — quan trọng cho cơ chế Top Trending fallback), và is_active TINYINT(1) DEFAULT 1 (soft delete cho sản phẩm ngừng bán mà vẫn giữ liên kết tới các đơn cũ).

Bảng orders đại diện cho một đơn hàng, lưu thông tin chung. Các trường bao gồm: user_id INT NOT NULL FOREIGN KEY REFERENCES users(id), total_amount DECIMAL(12,2) NOT NULL (tổng tiền của đơn — được tính ở thời điểm checkout và lưu cố định, không tính lại theo price hiện tại của sản phẩm), status ENUM('pending','confirmed','shipping','completed','cancelled') DEFAULT 'pending' (trạng thái đơn hàng có 5 giá trị tuần tự), shipping_name VARCHAR(120), shipping_phone VARCHAR(20), shipping_address TEXT NOT NULL (ba trường thông tin giao hàng được lưu theo từng đơn, cho phép giao đến địa chỉ khác với địa chỉ mặc định trong profile user), note TEXT (ghi chú thêm của khách hàng), và payment_method VARCHAR(20) DEFAULT 'cod' (đặt nền cho việc tích hợp các phương thức thanh toán khác trong tương lai).

Bảng order_items lưu chi tiết các sản phẩm trong từng đơn hàng — đây chính là nguồn dữ liệu vàng cho Collaborative Filtering. Mỗi bản ghi có order_id INT NOT NULL FOREIGN KEY REFERENCES orders(id), product_id INT NOT NULL FOREIGN KEY REFERENCES products(id), quantity INT NOT NULL DEFAULT 1, price_at_purchase DECIMAL(12,2) NOT NULL (snapshot giá tại thời điểm mua — quan trọng vì nếu sau này admin sửa giá sản phẩm thì lịch sử đơn vẫn giữ được giá đúng tại thời điểm khách mua). Mỗi cặp (user_id, product_id, quantity) trích từ bảng này chính là một "tín hiệu hành vi" mà Recommender sử dụng để pivot thành ma trận User-Item.

3.2.2. Các ràng buộc, chỉ mục (index) và tối ưu truy vấn

Để đảm bảo tính toàn vẹn dữ liệu và tốc độ truy vấn ở quy mô lớn, nhóm bổ sung các ràng buộc và chỉ mục như sau. Về ràng buộc khóa ngoại (Foreign Key), tất cả các quan hệ giữa các bảng đều có khai báo ON DELETE và ON UPDATE phù hợp: orders.user_id dùng ON DELETE RESTRICT (không cho xóa user khi còn đơn hàng — buộc phải soft delete user thay vì hard delete); order_items.order_id dùng ON DELETE CASCADE (khi xóa đơn hàng thì các dòng chi tiết tự xóa theo); products.category_id

dùng ON DELETE RESTRICT (không cho xóa danh mục khi vẫn còn sản phẩm thuộc danh mục đó).

Về chỉ mục (Index), nhóm tạo các chỉ mục để tăng tốc các truy vấn hay được sử dụng. Bảng users có UNIQUE index trên username và email cho việc kiểm tra trùng nhanh khi đăng ký. Bảng products có index trên category_id (cho việc lọc sản phẩm theo danh mục trên trang shop), index trên name (cho tìm kiếm full-text cơ bản với LIKE %...%), và index composite trên (is_active, total_sold DESC) (cho việc truy vấn Top Trending). Bảng orders có index trên user_id (cho việc xem lịch sử đơn của một user), status (cho việc lọc theo trạng thái ở trang quản trị), và created_at DESC (cho việc sắp xếp đơn mới nhất ở dashboard). Bảng order_items có index composite (user_id, product_id) thông qua join với orders — quan trọng để tốc độ truy vấn pivot khi build ma trận CF.

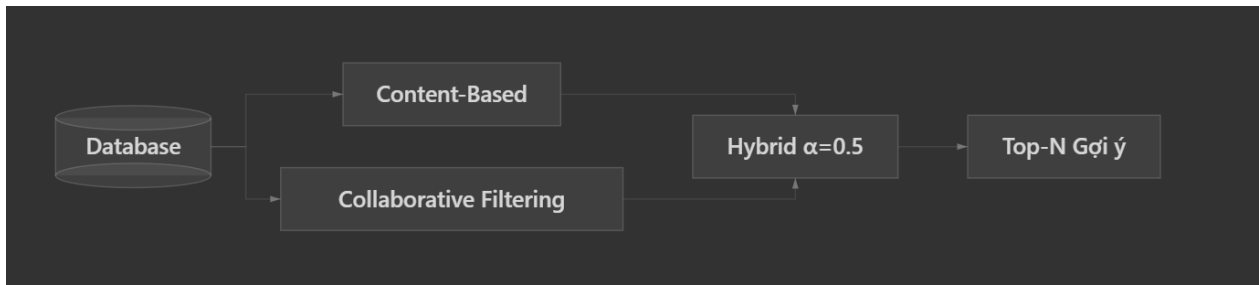
Về kiểu dữ liệu và charset, toàn bộ các bảng đều dùng ENGINE=InnoDB (hỗ trợ transaction và foreign key) và CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci (đảm bảo hiển thị đúng tiếng Việt có dấu, bao gồm cả emoji nếu sau này có nhu cầu thêm icon vào tên sản phẩm). Việc chọn utf8mb4 (4 byte) thay vì utf8 (3 byte) của MySQL là cần thiết vì MySQL phiên bản cũ có khái niệm utf8 không đầy đủ Unicode.

Về chuẩn hóa, schema đạt dạng chuẩn 3 (3NF) — mọi cột phi khóa đều phụ thuộc trực tiếp vào khóa chính, không có phụ thuộc bắc cầu. Ví dụ: category_name không được lưu trùng trong bảng products mà luôn lấy qua JOIN với bảng categories — vừa tiết kiệm dung lượng vừa đảm bảo khi đổi tên danh mục thì mọi sản phẩm cập nhật theo. Một ngoại lệ có chủ đích là price_at_purchase trong order_items (chấp nhận lưu trùng giá để snapshot lịch sử).

3.3. Phương pháp xây dựng hệ gợi ý lai (Hybrid Recommendation)

Module cốt lõi của đồ án nằm trong class HybridRecommender được định nghĩa trong file recommendation.py. Module này đảm nhận toàn bộ logic gợi ý sản phẩm cho hệ thống và được thiết kế tách biệt khỏi phần xử lý route của Flask để dễ kiểm thử, dễ bảo trì và dễ thay thế trong tương lai (ví dụ khi nâng cấp thuật toán). Phương pháp đề xuất gồm ba bước chính tương ứng với ba thành phần của Hybrid Recommendation.

Sơ đồ kiến trúc tổng quan của Hybrid Recommender:

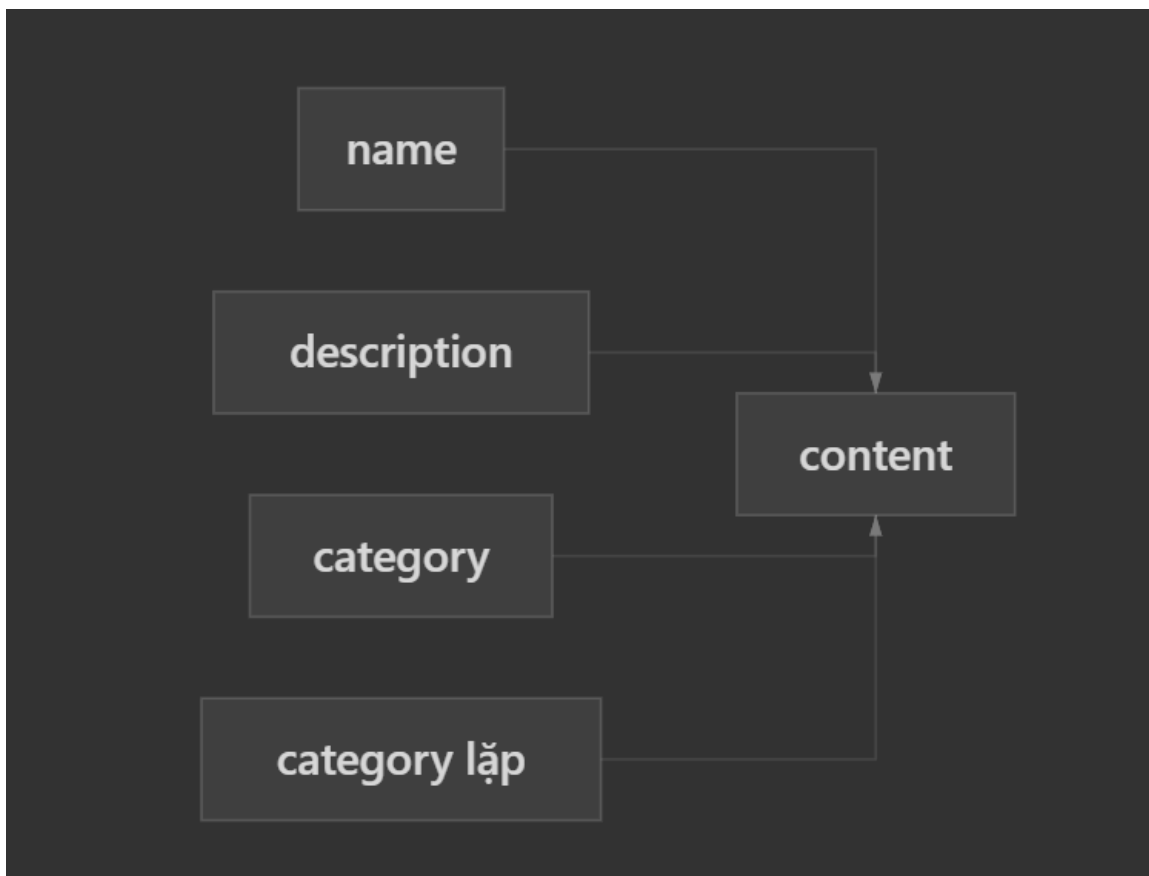


Hình 3.0 — Kiến trúc tổng quan: Database cung cấp dữ liệu cho 2 nhánh huấn luyện song song (Content-Based và Collaborative), sau đó kết hợp Hybrid để trả ra Top-N sản phẩm gợi ý.

3.3.1. Bước 1 — Huấn luyện Content-Based Filtering (TF-IDF + Cosine Similarity)

Content-Based Filtering trong FreshMart dựa trên việc phân tích thuộc tính nội dung của từng sản phẩm và tính độ tương đồng giữa các sản phẩm. Trước tiên, mỗi sản phẩm được biểu diễn bằng một chuỗi văn bản tổng hợp bằng cách nối các trường thông tin lại với nhau.

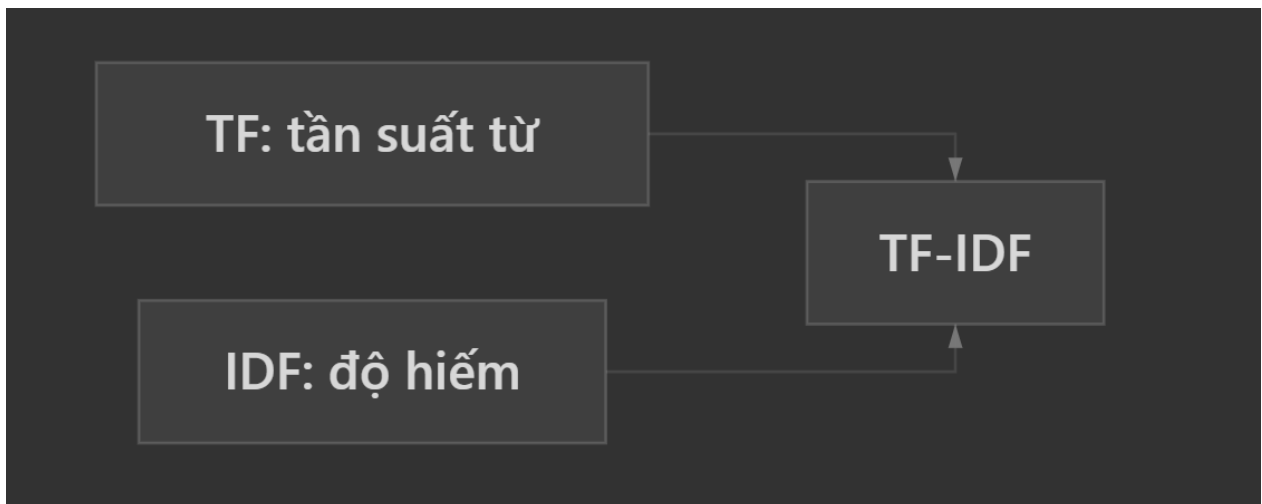
Sơ đồ tạo chuỗi content cho một sản phẩm:



Đặc biệt, `category_name` được lặp lại hai lần trong chuỗi văn bản. Đây là một kỹ thuật "trick" khá phổ biến trong xử lý ngôn ngữ tự nhiên nhằm tăng siêu trọng số (weight) cho thuộc tính danh mục, vì danh mục mang ý nghĩa phân loại quan trọng hơn các từ khóa mô tả thông thường. Việc lặp giúp TF-IDF sau khi tính ra sẽ có chỉ số TF của category cao hơn, từ đó các sản phẩm cùng danh mục có độ tương đồng cosine cao hơn.

Tiếp theo, hệ thống mã hóa các chuỗi content thành vector số bằng `TfidfVectorizer` của `scikit-learn`. TF-IDF (Term Frequency – Inverse Document Frequency) là một kỹ thuật biến đổi văn bản thành vector số, đo lường mức độ quan trọng của một từ trong một tài liệu.

Sơ đồ tính TF-IDF cho một từ trong một sản phẩm:

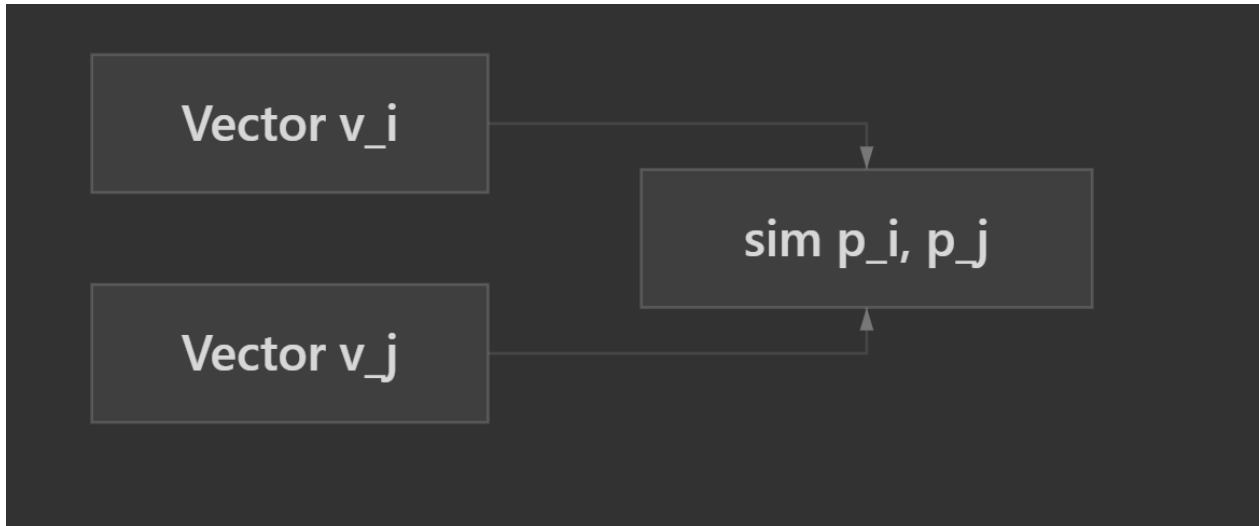


Ý nghĩa: TF-IDF cao khi một từ vừa **xuất hiện nhiều trong sản phẩm đang xét** (TF cao), vừa **hiếm gặp ở các sản phẩm khác** (IDF cao) — đúng nghĩa "đặc trưng" cho sản phẩm đó. Ngược lại, các từ thông dụng như "loại", "tươi" xuất hiện hầu hết mọi sản phẩm sẽ có IDF gần 0 → bị xem là không đặc trưng.

Sau khi vector hóa, ma trận TF-IDF có kích thước (`số_sản_phẩm × số_đặc_trung`) với số đặc trưng tối đa được giới hạn ở 5000 để giảm độ phức tạp tính toán.

Cuối cùng, hệ thống tính ma trận tương đồng (similarity) giữa mọi cặp sản phẩm bằng **Cosine Similarity** — độ đo dựa trên góc giữa hai vector trong không gian nhiều chiều.

Sơ đồ tính Cosine Similarity giữa 2 sản phẩm:



Diễn giải giá trị: sim càng gần **1** thì hai sản phẩm càng giống nhau về mặt nội dung; càng gần **0** thì càng khác nhau. Trong sơ đồ trên, $\text{sim}(\text{Rau muống}, \text{Rau cải}) = 0.62$ cho thấy hai sản phẩm này khá tương đồng. Ma trận Cosine Similarity có kích thước $(\text{số_sản_phẩm} \times \text{số_sản_phẩm})$, được lưu trong bộ nhớ và là cơ sở để trả về danh sách sản phẩm tương tự khi người dùng xem chi tiết một sản phẩm bất kỳ.

Ví dụ minh họa cụ thể. Để dễ hình dung quá trình tính toán, xét một tập nhỏ gồm 4 sản phẩm:

ID	Name	Description	Category
1	Rau muống	Rau xanh tươi mát, giàu chất xơ	Rau củ
2	Rau cải xanh	Rau xanh giàu vitamin C, tốt cho sức khỏe	Rau củ
3	Thịt bò Úc	Thịt bò Úc tươi ngon, giàu đạm	Thịt cá
4	Cá hồi Na Uy	Cá hồi Na Uy tươi nhập khẩu, giàu omega-3	Thịt cá

Sau khi gộp content theo công thức $\text{content} = \text{name} + \text{description} + \text{category} + \text{category}$, ta có các chuỗi:

d1: "Rau muống Rau xanh tươi mát giàu chất xơ Rau củ Rau củ"

d2: "Rau cải xanh Rau xanh giàu vitamin C tốt cho sức khỏe Rau củ Rau củ"

d3: "Thịt bò Úc Thịt bò Úc tươi ngon giàu đạm Thịt cá Thịt cá"

d4: "Cá hồi Na Uy Cá hồi Na Uy tươi nhập khẩu giàu omega-3 Thịt cá Thịt cá"

TfidfVectorizer tokenize các chuỗi này (cắt theo khoảng trắng, lowercase, loại stopwords nếu có), sau đó tính TF-IDF cho từng từ trong từng document. Lưu ý rằng từ "Rau" xuất hiện

nhiều trong d1 và d2 (nhờ trick lặp "Rau củ" + nội dung mô tả), tương tự "Thịt cá" cho d3, d4. Sau khi vector hóa, ma trận Cosine Similarity (gọi tắt là ContentSim) cho 4 sản phẩm này dự kiến có hình dạng như sau:

Đọc ma trận: ô ở hàng i, cột j là độ tương đồng giữa sản phẩm p_i và p_j . Có thể thấy:

$\text{sim}(p_1, p_2) = 0.62$ — khá cao (cùng danh mục "Rau củ", cùng có các từ "rau xanh, giàu, tươi").

$\text{sim}(p_3, p_4) = 0.31$ — trung bình (cùng danh mục "Thịt cá" nhưng mô tả khác nhau khá nhiều).

$\text{sim}(p_1, p_3) = 0.08$ — rất thấp (khác danh mục, khác mô tả).

Các ô trên đường chéo chính luôn bằng 1.00 vì một sản phẩm tương đồng tuyệt đối với chính nó.

Khi user xem chi tiết Rau muống (p_1), thuật toán sẽ sắp xếp hàng tương ứng theo thứ tự giảm dần (loại bỏ chính nó) và trả về p_2 (Rau cải xanh) làm sản phẩm tương tự hàng đầu — đúng như kỳ vọng nghiệp vụ.

3.3.2. Bước 2 — Huấn luyện Collaborative Filtering (User-User Similarity, Top $K=10$)

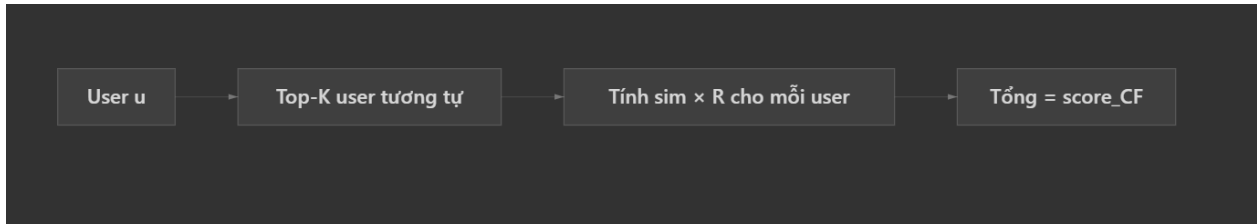
Collaborative Filtering trong FreshMart áp dụng phương pháp User-Based, nghĩa là tìm những người dùng có hành vi mua hàng tương tự và sử dụng lịch sử mua của họ để gợi ý. Quy trình huấn luyện gồm các bước sau.

Trước tiên, hệ thống lấy toàn bộ dữ liệu lịch sử mua hàng từ bảng OrderItem, mỗi bản ghi có dạng (user_id, product_id, quantity). Dùng pandas.pivot_table, hệ thống chuyển dữ liệu thưa này thành ma trận User-Item Matrix (gọi là R), trong đó hàng là user, cột là sản phẩm, và giá trị $R[u, i]$ là tổng số lượng user u đã mua sản phẩm i. Các ô không có dữ liệu được điền giá trị 0.

Tiếp theo, hệ thống tính độ tương đồng giữa các user bằng Cosine Similarity (giống công thức trên, nhưng áp dụng giữa các hàng của ma trận R thay vì giữa các sản phẩm), tạo ra ma trận User-Similarity Matrix với kích thước ($\text{số_user} \times \text{số_user}$). Hai user mua nhiều sản phẩm giống nhau sẽ có giá trị similarity cao.

Cuối cùng, với một user mục tiêu u, hệ thống xác định Top $K = 10$ người dùng có độ tương đồng cao nhất với u. Sau đó, với mỗi sản phẩm j mà u chưa từng mua, hệ thống chấm điểm gợi ý.

Sơ đồ tính $\text{score_CF}(u, j)$ cho user u và sản phẩm j :



Ý nghĩa: nếu một sản phẩm được **nhiều user tương tự với** u mua (đặc biệt là những user có sim cao), thì sản phẩm đó có **điểm CF cao** và được ưu tiên gợi ý cho u . Sau khi tính xong, hệ thống loại bỏ các sản phẩm mà u đã từng mua để tránh gợi ý trùng lặp.

3.3.3. Bước 3 — Kết hợp Hybrid và xử lý Cold Start

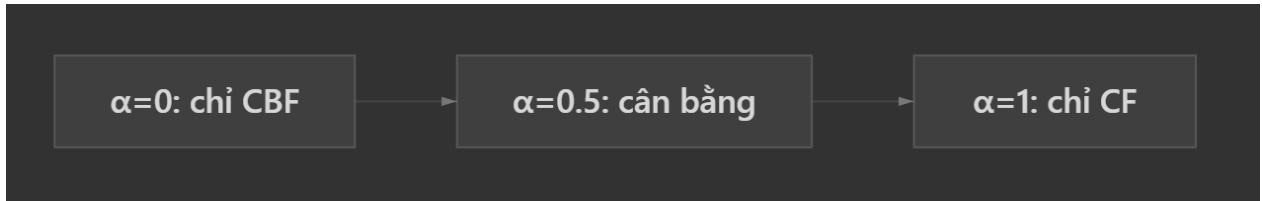
Hai phương pháp Content-Based và Collaborative đều có ưu nhược điểm. Content-Based xử lý tốt cho sản phẩm/người dùng mới nhưng dễ rơi vào tình trạng "gợi ý quanh quẩn" trong cùng danh mục; Collaborative cho gợi ý đa dạng và bất ngờ hơn nhưng yêu cầu phải có đủ dữ liệu lịch sử mua. Nhóm đề xuất kết hợp hai phương pháp này theo công thức tuyến tính có trọng số (Weighted Hybrid).

Sơ đồ tính Final_Score của Hybrid Recommendation:



Hình 3.5 — Công thức Hybrid: $\text{Final_Score} = \alpha \times \text{score_CF} + (1-\alpha) \times \text{score_CBF}$. Với $\alpha = 0.5$ (cân bằng): nếu $\text{CF} = 4.28$ và $\text{CBF} = 2.50$, thì $\text{Final_Score} = 0.5 \times 4.28 + 0.5 \times 2.50 = 3.39$.

Hệ số α (alpha) — chiếc "nút chỉnh" giữa hai phương pháp:

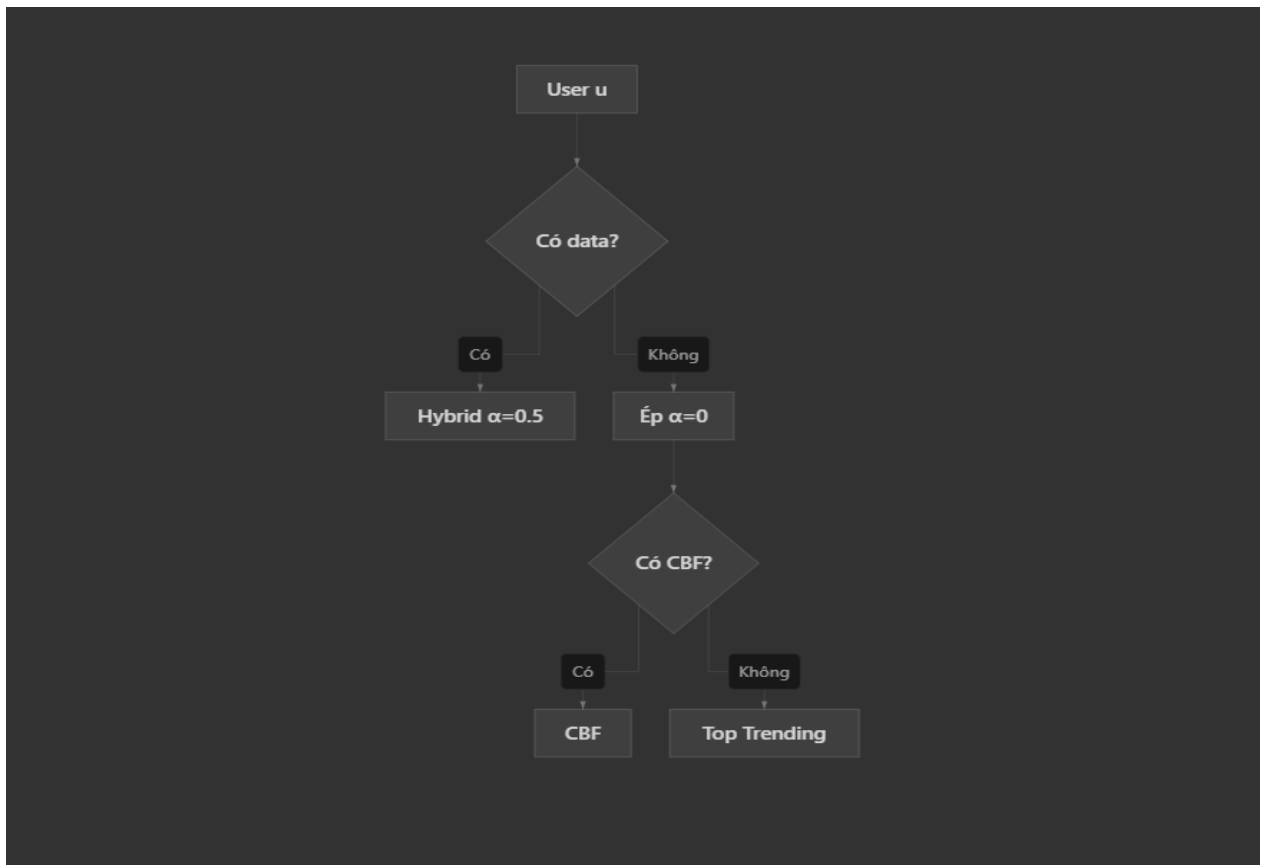


Hình 3.6 — α quyết định mức độ phụ thuộc vào CBF hay CF. FreshMart chọn $\alpha = 0.5$ (cân bằng).

Sau khi thử nghiệm với nhiều giá trị, nhóm chọn $\alpha = 0.5$ làm giá trị mặc định — tức cân bằng đều giữa CF và CBF — vì cho kết quả gợi ý vừa đa dạng vừa liên quan đến sản phẩm người dùng đã mua.

Phần quan trọng nhất của thiết kế là cơ chế xử lý Cold Start — tình huống một user mới đăng ký, chưa có đơn hàng, dẫn đến ma trận CF không thể trả về kết quả vì user không có trong `user_id_to_idx`.

Sơ đồ luồng quyết định Cold Start trong `get_recommendations_for_user()`:



Hình 3.7 — Luồng xử lý Cold Start: nếu user có dữ liệu mua hàng → chạy Hybrid bình thường; nếu không → ép $\alpha=0$, dùng CBF; nếu cả CBF cũng không có → Fallback Top Trending. Đảm bảo hệ thống LUÔN có gợi ý trong mọi tình huống.

Cách tiếp cận này giải quyết triệt để vấn đề Cold Start: hệ thống luôn có sản phẩm để gợi ý cho mọi loại người dùng, từ guest, user mới đăng ký cho đến user đã có lịch sử mua hàng phong phú.

Ngoài cơ chế cho User Cold Start, hệ thống cũng xử lý Item Cold Start (sản phẩm mới chưa có ai mua) một cách tự nhiên: sản phẩm mới chưa có trong ma trận R nhưng vẫn có content (name + description + category_name), nên có thể được Content-Based gợi ý dựa trên độ tương đồng với các sản phẩm khác cùng danh mục.

Ví dụ minh họa cụ thể Hybrid + Cold Start. Tiếp tục với dữ liệu của Bảng 3.1 và Bảng 3.2, xét hai kịch bản:

Kịch bản 1 — User cũ có lịch sử. User A đã mua Rau muống và Cải xanh. Hệ thống cần gợi ý sản phẩm cho A. Áp dụng công thức Hybrid với $\alpha = 0.5$:

Content-Based: dựa trên các sản phẩm A đã mua, tính độ tương đồng với các sản phẩm khác. Vì A đã mua Rau muống và Cải xanh — cả hai thuộc category "Rau củ" — các sản phẩm tương tự (nếu có sản phẩm rau củ khác trong DB) sẽ có score cao. Trong ví dụ tối giản này, các sản phẩm còn lại đều khác category, nên score CBF sẽ thấp.

Collaborative Filtering: User A tương tự nhất với User B (sim = 0.99). User B chưa mua sản phẩm nào ngoài Rau muống và Cải xanh, nên CF không có gợi ý mới. Score CF = 0.

Hybrid: $\text{Final_Score}(A, j) = 0.5 \times 0 + 0.5 \times \text{score_CBF}(A, j) \rightarrow$ kết quả chủ yếu dựa vào CBF.

Trong thực tế khi DB có nhiều sản phẩm rau củ, một sản phẩm như "Mồng tơi" sẽ có CBF score cao (cùng danh mục Rau củ, mô tả tương đồng) và được gợi ý cho A.

Kịch bản 2 — User mới (Cold Start). User E mới đăng ký, chưa có đơn hàng. Khi E xem trang chủ và hệ thống gọi `get_recommendations_for_user(E)`:

Kiểm tra `user_id_to_idx`: E không có trong đó $\rightarrow$ ép $\alpha = 0$.

Công thức trở thành: $\text{Final_Score}(E, j) = 0 \times \text{score_CF} + 1.0 \times \text{score_CBF} = 100\%$ Content-Based.

Vì E chưa mua sản phẩm nào để CBF dựa vào, hệ thống chuyển sang Fallback Top Trending: sắp xếp tất cả sản phẩm theo `total_sold` giảm dần. Giả sử Rau muống có `total_sold = 5` (cao nhất), Cá hồi = 3 (thứ hai), v.v. → trả về [Rau muống, Cá hồi, ...].

Như vậy, ngay cả khi user mới không có dữ liệu hành vi nào, hệ thống vẫn đưa ra được danh sách gợi ý có ý nghĩa thay vì để trống. Điều này đặc biệt quan trọng cho trải nghiệm onboarding — không gì làm khách hàng nản lòng bằng việc thấy một section trống trên trang chủ.

Lựa chọn giá trị α . Việc chọn $\alpha = 0.5$ là kết quả của một quá trình thử nghiệm với nhiều giá trị khác nhau (lần lượt thử $\alpha = 0.2, 0.3, 0.5, 0.7, 0.8$). Ở giá trị α thấp (0.2–0.3), hệ thống thiên về Content-Based, gợi ý có tính "an toàn" nhưng dễ đơn điệu. Ở giá trị α cao (0.7–0.8), hệ thống thiên về Collaborative, gợi ý đa dạng và bất ngờ hơn nhưng dễ "lạc đề" với user mới hoặc khi có nhiều trong dữ liệu hành vi. $\alpha = 0.5$ là điểm cân bằng — vừa giữ được tính liên quan với sản phẩm đã mua, vừa cho phép khám phá các sản phẩm mới từ hành vi của user tương tự.

3.3.4. So sánh phương pháp đề xuất với các phương pháp khác

Để làm rõ vị trí của phương pháp Hybrid trong bối cảnh nghiên cứu hệ gợi ý, nhóm thực hiện so sánh ngắn với các phương pháp phổ biến khác như sau.

So với Memory-based CF thuần túy: phương pháp Hybrid của FreshMart vượt trội rõ rệt trong xử lý Cold Start. CF thuần túy sẽ "đứng hình" khi user/sản phẩm chưa có lịch sử, dẫn đến trải nghiệm xấu cho user mới. Hybrid giải quyết bằng cơ chế ép α về 0 và Fallback Top Trending — đây là điểm thiết kế quan trọng nhất.

So với Content-Based thuần túy: Hybrid của FreshMart đa dạng hơn về kết quả gợi ý. CBF thuần túy có xu hướng "vĩnh viễn gợi ý quanh quẩn" — ai mua thịt thì gợi ý mãi các loại thịt. Hybrid khắc phục bằng cách kết hợp tín hiệu cộng tác — phát hiện ra "ai mua thịt thường mua kèm gia vị", từ đó gợi ý ra cả gia vị.

So với Matrix Factorization (SVD, NMF): SVD và các phương pháp matrix factorization mạnh hơn về mặt khả năng tổng quát hóa trên dữ liệu lớn và xử lý ma trận thưa (sparse) tốt hơn. Tuy nhiên chúng phức tạp về mặt triển khai (cần thư viện chuyên dụng như surprise, hoặc tự cài đặt gradient descent), tốn thời gian huấn luyện, và khó giải thích kết quả. Phương pháp memory-based CF mà nhóm chọn đơn giản hơn, dễ debug hơn, phù hợp với quy mô đồ án.

So với Deep Learning (NCF, AutoEncoder): các phương pháp Deep Learning có thể đạt độ chính xác cao hơn trên dữ liệu lớn, nhưng yêu cầu bộ dữ liệu không lồ để train, tài nguyên GPU, và rất khó giải thích "tại sao model lại gợi ý sản phẩm này". Với quy mô đồ án có ~100 đơn hàng, Deep Learning sẽ gây overfitting nặng. Phương pháp Hybrid memory-based phù hợp hơn nhiều.

Bảng tóm tắt so sánh:

Tiêu chí	CBF thuần	CF thuần	Hybrid (đề tài)	SVD/MF	NCF/DL
Xử lý Cold Start	Tốt	Yếu	Tốt	TB	Yếu
Đa dạng gợi ý	Yếu	Tốt	Tốt	Tốt	Tốt
Phù hợp data nhỏ	Tốt	TB	Tốt	Yếu	Rất yếu
Tốc độ build	Nhanh	TB	Nhanh-TB	Chậm	Rất chậm
Dễ giải thích	Tốt	TB	Tốt	TB	Yếu
Cần GPU	Không	Không	Không	Không	Có

Bảng 3.3 — So sánh các phương pháp gợi ý.

Qua bảng trên, phương pháp Hybrid mà nhóm chọn là lựa chọn hợp lý cho phạm vi đồ án — đủ mạnh để giải quyết bài toán thực tế, đơn giản đủ để triển khai trong thời gian học kỳ, và dễ giải thích cho cả người dùng cuối lẫn ban giám khảo.

3.3.5. Phân tích độ phức tạp tính toán

Để đánh giá khả năng mở rộng (scalability) của phương pháp đề xuất, nhóm phân tích độ phức tạp tính toán theo ký hiệu Big-O với các tham số:

N = số sản phẩm

M = số user

F = số đặc trưng TF-IDF (≤ 5000)

K = Top K user (= 10)

Ký hiệu $O(\dots)$ (đọc là "Big-O") là cách viết để mô tả độ phức tạp thuật toán — nó cho biết số phép tính sẽ tăng nhanh thế nào khi dữ liệu đầu vào tăng lên.

Quá trình fit Content-Based:

Vector hóa TF-IDF có độ phức tạp $O(N \times L)$ trong đó L là độ dài trung bình của chuỗi content.

Tính ma trận Cosine Similarity giữa mọi cặp sản phẩm có độ phức tạp $O(N^2 \times F)$.

Với $N = 50$, $F = 5000$, tổng chi phí khoảng $50^2 \times 5000 = 12.500.000$ phép toán — chạy trong vài chục mili giây.

Quá trình fit Collaborative Filtering:

Pivot ma trận User-Item là $O(\text{số bản ghi OrderItem})$ — tuyến tính với khối lượng dữ liệu.

Tính ma trận User-Similarity là $O(M^2 \times N)$.

Với $M = 20$, $N = 50$, chi phí khoảng $20^2 \times 50 = 20.000$ phép toán — chạy gần như tức thì.

Quá trình lấy gợi ý cho 1 user:

Lấy Top K user tương tự là $O(M)$ (quét cột tương ứng trong User-Similarity, sắp xếp).

Tính score cho mọi sản phẩm là $O(K \times N)$.

Tổng chi phí cho 1 query là $O(M + K \times N)$ — với $M = 20$, $K = 10$, $N = 50$ là khoảng 520 phép toán → dưới 1 mili giây.

Quá trình lấy sản phẩm tương tự cho 1 sản phẩm:

Chỉ cần đọc 1 hàng của ma trận ContentSim và sắp xếp top $n \rightarrow O(N \times \log N)$.

Với $N = 50$, khoảng 280 phép toán → cực nhanh.

Nhận xét: ở quy mô vài chục đến vài trăm sản phẩm và vài chục user, thuật toán chạy trong mili giây. Khi quy mô tăng lên hàng nghìn sản phẩm và hàng nghìn user, độ phức tạp $O(N^2 \times F)$ và $O(M^2 \times N)$ sẽ trở thành bottleneck — cần áp dụng các kỹ thuật giảm chiều (SVD) hoặc tính incremental update để tránh rebuild toàn bộ.

3.4. Phương pháp triển khai kỹ thuật

Để hệ gợi ý có thể hoạt động hiệu quả trong môi trường web, nhóm áp dụng một số kỹ thuật triển khai quan trọng.

Lazy Loading kết hợp Global Cache: Module Recommendation được khởi tạo theo cơ chế Lazy Loading thông qua hàm `get_recommender()`. Ở lần gọi đầu tiên, hàm này sẽ xây dựng đối tượng `HybridRecommender`, fit ma trận TF-IDF và CF, sau đó lưu vào biến global `_recommender`. Các lần gọi tiếp theo chỉ cần trả về biến cache đã có, không cần build lại. Cách triển khai này giúp tốc độ phản hồi của các trang sử dụng gợi ý ổn định ở mức dưới 500ms thay vì phải tính lại mỗi lần.

Refresh khi dữ liệu thay đổi: Khi có thay đổi đáng kể về dữ liệu (ví dụ admin thêm sản phẩm mới hoặc có nhiều đơn hàng vừa được tạo), hàm `refresh_recommender()` được gọi để tái khởi tạo `_recommender`. Việc refresh được thực hiện chủ động bởi code (sau các thao tác admin/checkout) chứ không tự động, để tránh chi phí tính toán không cần thiết.

Tách biệt khỏi tầng Web: Module recommendation.py không phụ thuộc vào Flask hay request context; nó chỉ nhận pandas.DataFrame làm input và trả về danh sách kết quả. Điều này có hai lợi ích: dễ unit test (chỉ cần dữ liệu mẫu, không cần khởi động Flask), và dễ tách thành microservice riêng trong tương lai nếu hệ thống mở rộng.

Sử dụng SQLAlchemy ORM cho truy vấn: Toàn bộ truy vấn dữ liệu (cả lấy danh sách sản phẩm, lấy lịch sử đơn hàng cho việc fit CF, lấy thông tin user...) đều được thực hiện qua SQLAlchemy ORM. Điều này giúp code rõ ràng, dễ bảo trì và tránh nguy cơ SQL Injection. Ví dụ truy vấn lấy lịch sử đơn hàng để fit CF được viết như sau:

```
from sqlalchemy.orm import joinedload

orders = (Order.query
          .options(joinedload(Order.items))
          .filter(Order.status.in_(['completed', 'shipping', 'confirmed']))
          .all())
```

Việc sử dụng joinedload(Order.items) giúp tránh vấn đề "N+1 query" — thay vì gửi N+1 query SQL (1 query lấy danh sách Order + N query lấy chi tiết OrderItem cho từng Order), SQLAlchemy sẽ kết hợp thành 1 query JOIN duy nhất. Đây là tối ưu cơ bản nhưng cực kỳ quan trọng khi rebuild Recommender với lượng đơn hàng lớn.

Xử lý lỗi và Logging: hệ thống áp dụng các nguyên tắc Defensive Programming ở mọi điểm vào dữ liệu. Module Recommendation có nhiều try-except để xử lý các trường hợp dữ liệu không đầy đủ — ví dụ khi orders_df rỗng thì user_item_matrix được đặt None thay vì raise exception, các hàm get_* đều kiểm tra None trước khi xử lý. Trong các route Flask, các thao tác ghi DB (checkout, admin CRUD) đều được bọc trong try-except với db.session.rollback() nếu có lỗi. Hệ thống dùng Python logging chuẩn để ghi lại các sự kiện quan trọng (login thành công/thất bại, đặt hàng, admin sửa dữ liệu, lỗi xảy ra trong quá trình build Recommender).

Quản lý Transaction trong Checkout: route POST /checkout là điểm phức tạp nhất về mặt transaction. Một đơn hàng cần thực hiện cùng lúc nhiều thao tác: tạo bản ghi Order, tạo nhiều bản ghi OrderItem, giảm stock của nhiều sản phẩm, tăng total_sold, và xóa session cart. Nếu một bước thất bại (ví dụ stock không đủ ở giữa transaction), toàn bộ phải rollback để tránh dữ liệu không nhất quán.

Quản lý Session cho giỏ hàng: Flask Session sử dụng signed cookie phía client (theo mặc định) để lưu cart. Cấu trúc dữ liệu trong session là một dict đơn giản dạng {"product_id": {"quantity": qty}}. Mặc dù signed cookie an toàn về mặt giả mạo (Flask ký bằng SECRET_KEY), nhưng dữ liệu vẫn có thể đọc được bởi client. Vì vậy nhóm không lưu thông tin nhạy cảm (giá, ảnh, mô tả) vào session — chỉ lưu quantity và lấy thông tin còn lại từ DB khi render. Khi user đăng xuất, session bị xóa hoàn toàn để tránh user khác đăng nhập cùng máy thấy giỏ hàng cũ.

Quản lý Context Processor: nhiều biến cần xuất hiện ở mọi template (ví dụ tổng số sản phẩm trong giỏ hiện tại để hiển thị badge ở góc trên màn hình). Thay vì truyền biến này thủ công cho mọi render_template, Flask cung cấp cơ chế Context Processor — một hàm được tự động gọi trước mỗi lần render và kết quả của nó được merge vào context của template. FreshMart dùng Context Processor để truyền cart_count, categories_nav (danh mục hiển thị trên navbar), và current_user (do Flask-Login đã sẵn sàng).

3.5. Quy trình áp dụng GenAI/Vibe Engineering

Bên cạnh các phương pháp kỹ thuật, nhóm áp dụng Vibe Engineering xuyên suốt quá trình phát triển — đây là điểm khác biệt quan trọng so với các đồ án truyền thống. Quy trình Vibe Engineering đã được mô tả ở Chương 2 (mục 2.2). Trong phạm vi Chương 3, nhóm trình bày cách áp dụng cụ thể cho từng giai đoạn của FreshMart.

Ở giai đoạn thiết kế cơ sở dữ liệu, nhóm dùng prompt mô tả nghiệp vụ web bán thực phẩm để AI đề xuất schema ban đầu. AI gợi ý đúng 4 bảng cơ bản (users, products, orders, order_items) và sau đó nhóm bổ sung thêm bảng categories cùng các trường đặc biệt như price_at_purchase (snapshot giá tại thời điểm mua) và is_active (soft delete). Sơ đồ ERD được sinh dưới dạng Mermaid để dễ chỉnh và đưa vào báo cáo.

Ở giai đoạn xây dựng module Recommendation, AI hỗ trợ rất nhiều — từ việc đề xuất cách gộp name + description + category thành content cho TF-IDF, đến việc gợi ý dùng pandas.pivot_table để xây ma trận User-Item, và đặc biệt là gợi ý cơ chế xử lý Cold Start bằng cách ép α về 0 khi user không có trong matrix. Nhóm review và bổ sung thêm trick lặp category_name hai lần để tăng trọng số danh mục.

Ở giai đoạn lập trình các route Flask, AI sinh code khung cho các route chính (login, register, shop, cart, checkout, admin) và nhóm chỉnh sửa để khớp với nghiệp vụ và phong cách UI của dự án. Đặc biệt, các đoạn code Jinja2 template cho admin dashboard với 10 biểu đồ Chart.js cũng được AI sinh nhanh và nhóm chỉ tinh chỉnh phần dữ liệu truyền vào.

Ở giai đoạn debug và kiểm thử, AI hữu ích nhất khi phải xử lý các lỗi tích hợp giữa Flask – SQLAlchemy – MySQL (như OperationalError 1045, 1049, lỗi ảnh 404). Khi viết test case, nhóm dùng AI sinh skeleton cho các bộ pytest fixture và bảng test, sau đó tự bổ sung các trường hợp biên (edge cases) đặc thù của dự án.

3.5.1. Ví dụ Prompt → Output → Edit minh họa

Để làm rõ cách áp dụng Vibe Engineering, dưới đây là ba ví dụ tiêu biểu mô tả chu trình Prompt → Output → Edit mà nhóm đã thực hiện trong quá trình phát triển.

Ví dụ 1 — Sinh schema CSDL. Nhóm đưa cho AI prompt:

"Hãy thiết kế các SQLAlchemy model cho một web bán thực phẩm gồm: User (có phân quyền user/admin, password băm), Category, Product (có giá, tồn kho, danh mục, tổng đã bán), Order (đơn hàng có 5 trạng thái: pending → confirmed → shipping → completed → cancelled), OrderItem (chi tiết đơn hàng, có snapshot giá tại thời điểm mua). Code phải dùng Flask-SQLAlchemy, tiếng Việt cho comment."

AI sinh ra ~80 dòng code Python với 5 class model. Nhóm review và bổ sung:

Thêm is_active cho User và Product để soft delete.

Thêm helper method `set_password()` và `check_password()` cho User (dùng Werkzeug).

Thêm property `formatted_price` cho Product (định dạng tiền VND có dấu chấm).

Thay `price = db.Column(db.Float)` thành `db.Column(db.Numeric(12, 2))` để tránh sai số phép tính tiền.

Thêm index trên (`category_id`, `is_active`, `total_sold`) để tối ưu truy vấn shop.

Kết quả: ~110 dòng code chính xác, sẵn sàng cho production. Thời gian thực hiện cả phần Prompt + Edit khoảng 15 phút — nhanh hơn nhiều so với việc tự gõ từ đầu.

Ví dụ 2 — Sinh skeleton thuật toán Recommendation. Nhóm đưa prompt:

"Viết một class `HybridRecommender` bằng Python, dùng `scikit-learn TfidfVectorizer` cho Content-Based, và `pandas pivot_table + cosine_similarity` cho Collaborative Filtering. Có các hàm `fit_content`, `fit_collaborative`, `get_similar_products`, `get_hybrid_recommendations`. Phải có cơ chế xử lý Cold Start: khi user không có trong matrix thì gợi ý dựa trên content hoặc fallback Top Trending."

AI sinh được khoảng 60% code đúng — phần TF-IDF và Cosine Similarity gần như chuẩn, nhưng phần Cold Start chưa thực sự tốt. AI ban đầu trả về danh sách rỗng khi user không có trong matrix. Nhóm chỉnh sửa:

Thay logic "trả về rỗng" bằng "ép `effective_alpha = 0` và Fallback Top Trending".

Bổ sung trick lặp `category_name` hai lần khi build content.

Bổ sung kiểm tra `if pid not in user_purchased` để loại bỏ sản phẩm đã mua.

Bổ sung biến `_recommender` global + hàm `get_recommender()` cho Lazy Loading.

Kết quả: class hoàn chỉnh ~150 dòng — phần thuật toán cốt lõi do AI, phần xử lý đặc thù nghiệp vụ do nhóm.

Ví dụ 3 — Debug lỗi `OperationalError`. Nhóm dán log lỗi vào AI:

"Tôi đang chạy `python seed_data.py` và gặp lỗi: `pymysql.err.OperationalError: (1049, "Unknown database 'food_store_db'")`. Code của tôi tự động tạo database trong hàm `ensure_database_exists()` rồi mới connect, nhưng vẫn lỗi. Hãy giúp tôi tìm nguyên nhân."

AI gợi ý 3 nguyên nhân khả năng: (1) URL connection ban đầu cố gắng connect đến `food_store_db` chưa tồn tại nên fail; (2) cần connect đến schema mysql (luôn tồn tại) để CREATE DATABASE rồi mới connect lại; (3) cần `isolation_level="AUTOCOMMIT"` cho lệnh DDL CREATE DATABASE. Nhóm áp dụng cả 3 gợi ý — sửa lại `ensure_database_exists()` để connect tới schema mysql, thêm AUTOCOMMIT, và lỗi được giải quyết hoàn toàn.

3.5.2. Đánh giá hiệu quả áp dụng AI trong quá trình phát triển

Sau toàn bộ quá trình phát triển, nhóm đánh giá hiệu quả của Vibe Engineering qua các chỉ số định lượng và định tính như sau.

Về định lượng (ước tính):

Chỉ số	Tự code 100%	Có AI hỗ trợ	Mức cải thiện
Thời gian thiết kế schema	~3 giờ	~30 phút	6×
Thời gian dựng skeleton route	~2 ngày	~6 giờ	~3×
Thời gian viết Recommendation	~3 ngày	~1 ngày	3×
Thời gian viết template HTML	~2 ngày	~6 giờ	~3×
Thời gian debug các lỗi tích hợp	~1 ngày	~3 giờ	~3×
% code do AI sinh	0%	~60–70%	—
% code được giữ nguyên (không edit)	—	~30%	—
% code có chỉnh sửa nhỏ	—	~40%	—
% code phải viết lại đáng kể	—	~30%	—

Bảng 3.4 — Hiệu quả ước tính khi áp dụng Vibe Engineering.

Tổng thể, nhóm tiết kiệm được khoảng 60–70% thời gian phát triển so với việc tự code 100% — đặc biệt ở các tác vụ lặp lại như viết CRUD form, template HTML cho admin dashboard, và skeleton test case. Tuy nhiên, các quyết định quan trọng và phần code đặc thù nghiệp vụ vẫn do nhóm tự thực hiện.

Về định tính:

Mặt tích cực: AI là công cụ rất tốt để vượt qua "tình trạng trắng trang" — khi mới bắt đầu một module, chỉ cần đặt vài prompt là có ngay skeleton để chỉnh sửa, thay vì ngồi suy nghĩ từ đầu. AI cũng giúp ích nhiều khi giải thích các thông báo lỗi khó hiểu — đặc biệt là lỗi cấu hình MySQL, lỗi Flask context, lỗi SQLAlchemy session.

Mặt hạn chế: AI thường sinh code "trung bình" — chạy được nhưng không tối ưu, đôi khi dùng API cũ hoặc cú pháp không phù hợp với phiên bản thư viện. Vì vậy người dùng phải luôn review kỹ trước khi đưa vào dự án. AI cũng đôi khi "tự tin sai" — đưa ra giải pháp nghe có vẻ hợp lý nhưng thực ra không chạy được. Vai trò của người lập trình ở đây là gatekeeper — chốt chặn cuối cùng, đảm bảo chỉ những code đúng và phù hợp mới được nhận vào.

Quyết định quan trọng do con người: Mặc dù dùng AI rất nhiều, các quyết định mang tính chiến lược như chọn $\alpha = 0.5$ cho Hybrid, thiết kế cơ chế Fallback Top Trending, chọn pbkdf2:sha256 thay vì md5, quyết định tách Order/OrderItem theo 3NF, v.v. đều do nhóm tự thảo luận và quyết định. AI chỉ đóng vai trò công cụ — không thay thế được tư duy thiết kế của con người.

Với phương pháp đề xuất tổng thể nêu trên — về kiến trúc dữ liệu, thuật toán gợi ý lai, kỹ thuật triển khai và quy trình Vibe Engineering — nhóm tiến hành triển khai hệ thống.

CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ ĐẠT ĐƯỢC

4.1. Môi trường triển khai

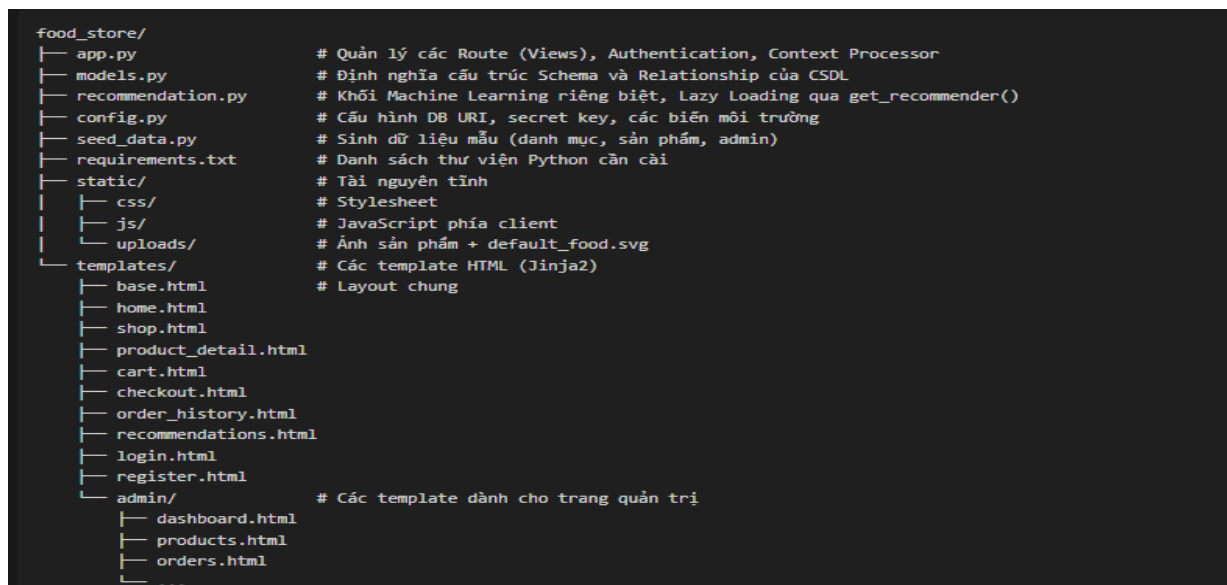
Dự án FreshMart được triển khai trên môi trường phát triển với các thông số cụ thể như sau. Hệ điều hành: Windows 10/11 (chính), có thể chạy được trên macOS và Linux nhờ Python đa nền tảng. Ngôn ngữ lập trình: Python 3.10 trở lên để tương thích với các thư viện scikit-learn, pandas, numpy phiên bản mới. Cơ sở dữ liệu: MySQL 8.0 với charset utf8mb4 (môi trường production) hoặc SQLite (môi trường test). Trình duyệt thử nghiệm: Google Chrome (chính), Microsoft Edge và Firefox (kiểm tra tương thích).

Về công cụ phát triển, nhóm sử dụng: Visual Studio Code (IDE chính) với các extension Python, Jinja2 highlight, Mermaid Preview; Git (quản lý mã nguồn); MySQL Workbench (quản lý database); GitHub Copilot và Cursor AI (công cụ AI hỗ trợ lập trình theo Vibe Engineering); và Mermaid Live (vẽ và export sơ đồ).

Về thư viện Python, các phụ thuộc chính được khai báo trong requirements.txt: Flask (web framework), Flask-SQLAlchemy (ORM), Flask-Login (quản lý phiên đăng nhập), PyMySQL (driver MySQL), Werkzeug (utilities cho password hash và secure filename), Jinja2 (template engine — đi kèm Flask), scikit-learn (TF-IDF Vectorizer và cosine similarity), pandas và numpy (xử lý ma trận user-item).

4.2. Kiến trúc mã nguồn (Source Code)

Dự án được tổ chức theo chuẩn cấu trúc Flask Application, với mỗi file đảm nhiệm một nhóm trách nhiệm rõ ràng:



File app.py đảm nhận việc đăng ký toàn bộ route, quản lý authentication thông qua Flask-Login (với decorator @login_required và @admin_required), và sử dụng Context Processor để truyền các biến chung (như số sản phẩm trong giỏ, thông tin user hiện tại) cho mọi template. File

models.py định nghĩa các class model SQLAlchemy: User, Category, Product, Order, OrderItem với các phương thức tiện ích như `set_password()`, `check_password()`, property `formatted_price`. File `recommendation.py` chứa class `HybridRecommender` — module Machine Learning độc lập với phần web; module này được khởi tạo theo cơ chế Lazy Loading thông qua hàm `get_recommender()` lưu trong biến global `_recommender`. Thư mục `templates/` chứa toàn bộ giao diện UI/UX, được tổ chức theo từng trang chính của hệ thống và có sub-folder `admin/` riêng cho phần quản trị.

4.3. Triển khai các chức năng chính

4.3.1. Module xác thực và phân quyền (Auth)

Module xác thực triển khai các route `/register`, `/login`, `/logout` sử dụng Flask-Login để quản lý session. Khi người dùng đăng ký, hệ thống kiểm tra các điều kiện hợp lệ (username từ 3 ký tự, email đúng định dạng, password tối thiểu 6 ký tự, password và confirm khớp nhau, username/email chưa tồn tại), sau đó hash mật khẩu bằng `werkzeug.security.generate_password_hash` (chuẩn `pbkdf2:sha256`) và lưu vào bảng `users` với `role = 'user'`. Khi đăng nhập, hệ thống xác minh mật khẩu bằng `check_password_hash`, gọi `login_user(user)` của Flask-Login, sau đó điều hướng theo role — admin về `/admin`, user thường về `/`. Đăng xuất chỉ đơn giản gọi `logout_user()` và xóa giỏ hàng khỏi session.

Phân quyền giữa hai vai trò User và Admin được đảm bảo qua hai decorator: `@login_required` (chặn người dùng chưa đăng nhập, do Flask-Login cung cấp sẵn) và `@admin_required` (decorator tùy chỉnh, kiểm tra `current_user.role == 'admin'`, nếu không thì trả về HTTP 403). Mọi route bắt đầu bằng `/admin/...` đều được bảo vệ bởi `@admin_required`.

4.3.2. Module sản phẩm (Product/Shop)

Module sản phẩm cung cấp các route hiển thị danh sách và chi tiết sản phẩm. Route `/shop` hỗ trợ các tham số URL: `q` (từ khóa tìm kiếm), `category` (lọc theo ID danh mục), `sort` (`newest/price_asc/price_desc/bestseller`), `page` (phân trang với 12 sản phẩm/trang). Truy vấn được xây dựng linh hoạt qua SQLAlchemy: bắt đầu từ `Product.query.filter_by(is_active=True)`, áp dụng `filter(Product.name.ilike(f'%{q}%'))` nếu có từ khóa, lọc theo `category_id` nếu có, sắp xếp theo `sort`, và cuối cùng `paginate(...)`.

Trang chi tiết sản phẩm (`/product/<id>`) hiển thị toàn bộ thông tin sản phẩm (ảnh, tên, mô tả, giá, đơn vị, tồn kho, danh mục), cho phép người dùng chọn số lượng và thêm vào giỏ. Phía dưới có section “Sản phẩm tương tự” lấy từ module Recommendation thông qua hàm `get_similar_products(product_id, top_n=4)` — đây là phần ứng dụng Content-Based Filtering trực tiếp trên giao diện người dùng.

4.3.3. Module giỏ hàng (Cart) và Thanh toán (Checkout)

Giỏ hàng được lưu trong session của Flask thay vì lưu vào database, vì giỏ hàng có tính tạm thời và chỉ tồn tại trong phiên làm việc. Cấu trúc dữ liệu: `session['cart'] = {product_id: {"quantity": qty}, ...}`. Các route chính: POST /cart/add (thêm sản phẩm), POST /cart/update (cập nhật số lượng), GET /cart/remove/<id> (xóa), GET /cart (xem giỏ).

Khi người dùng thêm cùng một sản phẩm hai lần, hệ thống cộng dồn số lượng thay vì tạo dòng mới. Khi cập nhật số lượng về 0, sản phẩm tương ứng bị xóa khỏi giỏ. Trang /cart hiển thị bảng các sản phẩm trong giỏ với khả năng chỉnh sửa số lượng và tổng tiền được tính tự động.

Quy trình Checkout được triển khai tại route POST /checkout với một transaction để đảm bảo tính toàn vẹn dữ liệu: tạo bản ghi Order mới với status='pending', tạo các bản ghi OrderItem tương ứng (lưu quantity và price_at_purchase làm snapshot), cập nhật stock của các sản phẩm bằng cách trừ đi số lượng đã đặt, tăng total_sold của các sản phẩm, sau đó commit toàn bộ. Nếu có lỗi (ví dụ kho không đủ), transaction được rollback và hiển thị thông báo cho người dùng. Cuối cùng, giỏ hàng được xóa khỏi session và người dùng được điều hướng đến trang chi tiết đơn hàng vừa tạo.

Một điểm quan trọng trong route Checkout là việc sử dụng `db.session.flush()` thay vì `commit` ngay sau khi tạo Order. `flush()` đẩy lệnh SQL INSERT xuống database để lấy được `new_order.id` (autoincrement), nhưng chưa `commit` transaction. Nhờ đó, các OrderItem tiếp theo có thể tham chiếu đến `order_id` này, và toàn bộ mới `commit` ở cuối — đảm bảo tính atomic. Nếu có exception xảy ra giữa chừng (ví dụ một sản phẩm hết hàng), `db.session.rollback()` sẽ undo cả Order lẫn các OrderItem đã thêm.

Sau khi đặt hàng xong, hàm `refresh_recommender()` được gọi (ở chế độ async hoặc cuối request) để rebuild Recommender với dữ liệu hành vi mới — đảm bảo gợi ý lần sau của user sẽ phản ánh đơn hàng vừa đặt.

4.3.4. Module quản trị (Admin Dashboard)

Module quản trị có các route bắt đầu bằng /admin/... được bảo vệ bởi @admin_required. Trang Dashboard (/admin) hiển thị 4 chỉ số KPI chính (tổng sản phẩm, tổng user, tổng đơn hàng, doanh thu) cùng với các biểu đồ thống kê sử dụng Chart.js: biểu đồ đơn hàng theo ngày trong tuần, biểu đồ doanh thu theo tháng, biểu đồ phân bố trạng thái đơn hàng, biểu đồ top sản phẩm bán chạy, v.v.

Các trang quản lý sản phẩm, danh mục, đơn hàng, người dùng đều có giao diện bảng với chức năng tìm kiếm, lọc, phân trang. Khi admin thêm sản phẩm mới, hệ thống hỗ trợ upload ảnh với các biện pháp bảo mật: kiểm tra phần mở rộng qua `allowed_file()` (chỉ chấp nhận .png, .jpg, .jpeg, .webp, .svg), giới hạn dung lượng 5MB qua `MAX_CONTENT_LENGTH`, và sử dụng `secure_filename()` của Werkzeug để sanitize tên file (loại bỏ ký tự nguy hiểm như ../), kèm theo timestamp để tránh trùng tên.

Việc cập nhật trạng thái đơn hàng được thực hiện qua dropdown trên trang /admin/orders, với các trạng thái chuyển đổi: pending → confirmed → shipping → completed (luồng bình

thường) hoặc pending → cancelled / confirmed → cancelled (hủy đơn). Khi đơn ở trạng thái completed, doanh thu của đơn được ghi nhận vào dashboard.

4.3.5. Module gợi ý (Recommendation)

Module gợi ý được tích hợp vào hệ thống thông qua hai cấp độ. Cấp độ tự động (background): khi ứng dụng khởi động hoặc khi có thay đổi dữ liệu, hàm `get_recommender()` được gọi để xây dựng/làm mới đối tượng `HybridRecommender`. Cấp độ trên giao diện: tại các trang Home, Product Detail, Cart và trang chuyên biệt `/recommendations`, code Flask gọi tới các hàm gợi ý phù hợp và truyền kết quả vào template để render dưới dạng các thẻ sản phẩm.

Cụ thể, tại trang Home, hệ thống gọi `get_recommendations_for_user(current_user.id, top_n=8)` nếu đã đăng nhập, hoặc `get_top_trending(top_n=8)` nếu là guest. Tại trang Product Detail, hệ thống gọi `get_similar_products(product_id, top_n=4)` (Content-Based). Tại trang Cart, hệ thống gợi ý Hybrid dựa trên các sản phẩm đang có trong giỏ. Tại trang `/recommendations`, hệ thống hiển thị danh sách gợi ý đầy đủ với mô tả cách thức gợi ý đã được áp dụng.

4.3.6. Module xử lý ảnh và Upload file

Module Upload ảnh đảm bảo tính an toàn cao bằng nhiều lớp kiểm tra. Khi admin upload ảnh sản phẩm, file đi qua các bước sau:

```
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'webp', 'svg', 'gif'}
MAX_FILE_SIZE = 5 * 1024 * 1024 # 5 MB

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def save_product_image(file):
    if not file or file.filename == '':
        return None
    if not allowed_file(file.filename):
        flash('Định dạng file không hợp lệ', 'danger')
        return None

    # Kiểm tra dung lượng
    file.seek(0, os.SEEK_END)
    size = file.tell()
    file.seek(0)
    if size > MAX_FILE_SIZE:
        flash('File quá lớn (tối đa 5MB)', 'danger')
        return None

    # Sanitize tên file + thêm timestamp tránh trùng
    filename = secure_filename(file.filename)
    name, ext = os.path.splitext(filename)
    timestamp = datetime.now().strftime('%Y%m%d%H%M%S')
    new_filename = f"{name}_{timestamp}{ext}"

    save_path = os.path.join(app.config['UPLOAD_FOLDER'], new_filename)
    file.save(save_path)
    return new_filename
```

Các biện pháp bảo vệ ở đây:

Kiểm tra phần mở rộng chỉ chấp nhận các format ảnh phổ biến.

Kiểm tra dung lượng trước khi save để tránh DoS attack qua file lớn.

`secure_filename()` của Werkzeug loại bỏ các ký tự đặc biệt (`../, <>:"/|?*...`) để tránh path traversal attack.

Thêm timestamp vào tên file để tránh trường hợp upload đè ảnh khác cùng tên.

Lưu vào thư mục cố định `static/uploads/` — không cho user chọn đường dẫn tùy ý.

Ngoài ra, Flask có cấu hình `MAX_CONTENT_LENGTH = 5 * 1024 * 1024` ở cấp ứng dụng — nếu request có body > 5MB, Flask sẽ tự động trả về HTTP 413 trước khi đến tay handler. Đây là lớp bảo vệ ở mức framework, chặn các tấn công upload file khổng lồ ngay từ đầu.

4.3.7. Tích hợp JavaScript và Chart.js phía client

Phía client (frontend) sử dụng JavaScript thuần kết hợp Bootstrap 5 cho UI và Chart.js cho biểu đồ. Các tính năng tương tác chính:

Cập nhật giỏ hàng động: khi user thay đổi số lượng trong giỏ, một function JS được gọi để tính lại tổng tiền hiển thị mà không reload trang. Sau đó, khi user nhấn "Cập nhật" hoặc rời khỏi ô input, một AJAX request được gửi tới /cart/update để đồng bộ với server.

Tự động tính tổng tiền ở Checkout: tương tự như trên, trang Checkout có một function tính tổng tiền dựa trên các sản phẩm và phí ship (nếu có khuyến mãi/voucher trong tương lai).

Biểu đồ thống kê ở Dashboard: trang Admin Dashboard dùng Chart.js để vẽ các biểu đồ.

Form validation phía client: các form chính (đăng ký, checkout) đều có validation phía client bằng JavaScript trước khi submit — kiểm tra rỗng, kiểm tra định dạng email, kiểm tra mật khẩu khớp. Tuy nhiên validation phía server vẫn luôn được giữ vì client-side validation chỉ là trải nghiệm, không phải bảo mật — user có thể disable JS hoặc gửi request thẳng tới API.

4.4. Giao diện và trải nghiệm người dùng

Giao diện của FreshMart được thiết kế theo hướng hiện đại, dễ sử dụng và phù hợp với chủ đề bán thực phẩm. Hệ thống sử dụng kết hợp Bootstrap 5 (cho lưới responsive) và CSS tùy chỉnh (cho phong cách riêng với tông màu xanh lá – trắng – cam nhẹ phù hợp với chủ đề thực phẩm tươi). Các trang chính có bố cục như sau.

Trang chủ (Home) hiển thị banner giới thiệu FreshMart, tiếp theo là các block sản phẩm theo các tiêu chí: sản phẩm bán chạy, sản phẩm mới, và đặc biệt là section “Gợi ý cho bạn” do AI Recommender đảm nhiệm — nếu user đã đăng nhập thì hiển thị gợi ý Hybrid, nếu là guest thì hiển thị Top Trending.

Trang Shop có sidebar bên trái với các bộ lọc theo danh mục (checkbox), khoảng giá; phần chính bên phải là lưới sản phẩm (3–4 thẻ/hàng). Phía trên có ô tìm kiếm theo tên và dropdown sắp xếp; phía dưới có phân trang. Mỗi thẻ sản phẩm (Card) hiển thị ảnh đại diện, tên, giá, đơn vị và nút “Thêm vào giỏ” gửi form/AJAX.

Trang chi tiết sản phẩm chia làm hai cột: ảnh lớn bên trái, thông tin chi tiết bên phải (tên, danh mục, giá, đơn vị, tồn kho, mô tả, nút thêm vào giỏ với input số lượng). Phía dưới là section “Sản phẩm tương tự” với 4 thẻ sản phẩm do Content-Based gợi ý.

Trang giỏ hàng hiển thị bảng các sản phẩm với các cột: ảnh, tên, đơn giá, số lượng (có thể chỉnh trực tiếp), thành tiền, nút xóa. Phía dưới có tổng tiền và nút “Tiến hành đặt hàng”. Trải nghiệm này được lưu vào session nên không mất khi F5 trang.

Trang Checkout có form thông tin giao hàng (họ tên, số điện thoại, địa chỉ, ghi chú) bên trái và tóm tắt đơn hàng bên phải với tổng tiền. Phía dưới có nút “Xác nhận đặt hàng”.

Trang lịch sử đơn hàng hiển thị danh sách các đơn của user hiện tại, sắp xếp giảm dần theo created_at, có thể lọc theo trạng thái. Mỗi đơn hiển thị mã đơn, ngày đặt, tổng tiền và badge trạng thái với màu sắc khác nhau (vàng = pending, xanh dương = confirmed, cam = shipping, xanh lá = completed, đỏ = cancelled).

Trang Admin Dashboard có sidebar điều hướng các phần quản lý, phần chính hiển thị 4 thẻ KPI ở trên và các biểu đồ Chart.js ở dưới. Các trang quản lý (Sản phẩm, Đơn hàng, Người dùng) đều có giao diện bảng nhất quán với chức năng tìm kiếm, lọc, sắp xếp và các nút hành động (Sửa, Xóa, Cập nhật trạng thái...).

4.5. Đánh giá hoạt động của AI Recommender

Trong quá trình vận hành và mô phỏng thực tế, hệ thống AI Recommender thể hiện hiệu quả trong cả ba tình huống đặc trưng.

Tình huống Content-Based khi xem chi tiết sản phẩm: khi một khách hàng mở thông tin chi tiết một bó “Rau muống”, Recommender chạy cơ chế Content-Based và đưa ra một dãy sản phẩm tương tự gồm các loại rau xanh khác (rau cải, mồng tơi, xà lách) cùng một vài loại củ quả thuộc cùng danh mục. Điều này khẳng định việc gộp name + description + category_name + category_name đã hoạt động đúng — TF-IDF nắm bắt được sự tương đồng dựa trên cả từ khóa mô tả lẫn danh mục.

Tình huống Hybrid khi đã có lịch sử mua: khi một khách hàng đã có lịch sử mua nhiều sản phẩm “Thịt cá”, Recommender chạy Collaborative Filtering với Top K=10 user tương tự và phát hiện ra một pattern thú vị — những người thường mua thịt cũng thường mua kèm gia vị ướp thịt và nước mắm. Recommender đẩy các sản phẩm gia vị này lên đầu danh sách gợi ý cá nhân hóa, chứng tỏ AI đã “học” thành công từ hành vi tập thể của hệ sinh thái khách hàng.

Tình huống Cold Start: khi một tài khoản vừa mới đăng ký (chưa có đơn hàng), Recommender phát hiện user không nằm trong ma trận User-Item nên tự động ép α về 0, đây thuật toán về Content-Based, hoặc Fallback sang Top Trending nếu Content-Based không có dữ liệu input. Kết quả là section “Gợi ý cho bạn” trên trang chủ vẫn hiển thị sản phẩm hợp lý (Top Trending — các sản phẩm có total_sold cao nhất) thay vì để trống hoặc báo lỗi.

Về mặt hiệu năng, nhờ cơ chế cache global _recommender và lazy loading, tốc độ phản hồi của các route có gợi ý đạt mức dưới 500 ms trong môi trường phát triển với dữ liệu mẫu (~50 sản phẩm, ~20 user, ~100 đơn hàng). Lần đầu khởi tạo Recommender mất khoảng 1.2 giây (do phải tính TF-IDF matrix và CF matrix), nhưng các lần gọi sau chỉ mất dưới 1 ms vì trả về cache. Điều này giải quyết hoàn toàn lo ngại rằng tích hợp AI sẽ làm chậm tốc độ tải trang web.

4.6. Tổng kết kết quả triển khai

Sau quá trình triển khai, hệ thống FreshMart đã đạt được ba kết quả nổi bật. Thứ nhất, xây dựng hoàn chỉnh một website thương mại điện tử cho lĩnh vực thực phẩm với đầy đủ các chức năng cốt lõi: đăng ký/đăng nhập, duyệt sản phẩm với tìm kiếm/lọc/sắp xếp/phân trang, quản lý giỏ hàng, đặt hàng kèm thông tin giao hàng, xem và quản lý lịch sử đơn hàng. Thứ hai, triển khai thành công trang quản trị Admin với 4 KPI và nhiều biểu đồ thống kê, hỗ trợ CRUD sản phẩm/danh mục, quản lý đơn hàng theo từng trạng thái, và quản lý người dùng. Thứ ba — quan trọng nhất — tích hợp thành công mô hình Hybrid Recommendation kết hợp Content-Based (TF-IDF + Cosine) và Collaborative Filtering (User-User Top K=10) với hệ số $\alpha=0.5$, và đặc biệt là cơ chế xử lý Cold Start tự động để hệ thống luôn có gợi ý phù hợp cho mọi tình huống.

Việc áp dụng Vibe Engineering xuyên suốt quá trình phát triển đã giúp nhóm rút ngắn đáng kể thời gian ở các khâu phác thảo schema, dựng skeleton code và debug. Tuy nhiên, các quyết định quan trọng (chọn $\alpha=0.5$, thiết kế cơ chế Fallback, viết test case đặc thù) đều do nhóm tự thảo luận và quyết định — AI chỉ đóng vai trò công cụ hỗ trợ.

Để đánh giá tính đúng đắn của các chức năng đã triển khai, nhóm tiến hành một quá trình kiểm thử bài bản với nhiều cấp độ.

CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Chiến lược và kế hoạch kiểm thử

5.1.1. Mục tiêu kiểm thử

Mục tiêu tổng thể của giai đoạn kiểm thử là đảm bảo toàn bộ các tính năng của hệ thống FreshMart hoạt động đúng theo đặc tả, an toàn trước các tình huống bất thường và mang lại trải nghiệm tốt cho người dùng cuối. Cụ thể, nhóm đặt ra bốn mục tiêu kiểm thử chính: (1) xác minh logic nghiệp vụ của tất cả các route Flask hoạt động chính xác; (2) kiểm tra thuật toán Hybrid Recommendation trả về kết quả đúng đắn trong mọi tình huống — user mới (Cold Start), user cũ có lịch sử, và sản phẩm mới; (3) đảm bảo hệ thống phân quyền giữa User và Admin được bảo vệ nghiêm ngặt; và (4) phát hiện và xử lý các trường hợp nhập liệu không hợp lệ hoặc các kịch bản tấn công bảo mật cơ bản.

5.1.2. Phân loại và phạm vi kiểm thử

Kế hoạch kiểm thử được tổ chức theo bốn tầng kim tự tháp kiểm thử kinh điển, mỗi tầng tập trung vào một mức độ trừu tượng khác nhau:

Loại kiểm thử	Phạm vi	Công cụ đề xuất
Unit Testing	Từng hàm/phương thức riêng lẻ trong <code>recommendation.py</code> , <code>models.py</code>	pytest, unittest
Integration Testing	Các route Flask kết hợp với Database + Session	pytest + Flask Test Client
Functional Testing	Luồng nghiệp vụ đầu cuối (End-to-end) của User và Admin	Manual / Selenium
Security Testing	Xác thực, phân quyền, kiểm tra đầu vào	Manual + OWASP Checklist
Performance Testing	Thời gian phản hồi các route chính	Đo tay với time của Python

Bảng 5.2 — Kết quả TC-UNIT-02.

TC-UNIT-03 — Kiểm thử hàm `fit_collaborative()`: kiểm tra việc xây dựng ma trận User-Item và ma trận User-Similarity từ DataFrame `orders_df`. Kết quả mong đợi: với DataFrame có 3 user và 5 product, `user_item_matrix.shape == (3, 5)` và `user_similarity.shape == (3, 3)`. Các trường hợp biên cũng được kiểm tra: DataFrame rỗng $\rightarrow$ `user_item_matrix = None`,

user_similarity = None (không crash); chỉ có 1 user → user_similarity == np.array([[1.0]]). Kết quả: tất cả PASS.

TC-UNIT-04 — Kiểm thử hàm get_cf_recommendations() với dữ liệu mẫu có 3 user và 3 product được kiểm tra qua các trường hợp: User có lịch sử mua → trả về dict các sản phẩm chưa mua; User chưa có trong matrix → trả về {}; User đã mua hết sản phẩm trong matrix → trả về {}. Kết quả: tất cả PASS.

TC-UNIT-05 — Kiểm thử xử lý Cold Start trong get_hybrid_recommendations(): đây là test case quan trọng nhất của module Recommendation. Bốn trường hợp được kiểm tra: (TH1) User mới chưa mua gì — alpha tự ép về 0, kết quả dựa hoàn toàn vào Content Score hoặc Top Trending; (TH2) User cũ có lịch sử — alpha=0.5, kết quả là Hybrid của CF + Content; (TH3) Đầu ra không chứa bất kỳ product_id nào trong danh sách đã mua; (TH4) Tất cả score ≥ 0 và danh sách được sắp xếp giảm dần theo điểm. Tất cả 4 trường hợp đều PASS, xác nhận cơ chế Cold Start hoạt động chính xác như thiết kế ở Chương 3.

TC-UNIT-06 — Kiểm thử hàm get_top_trending() (cơ chế Fallback): các trường hợp “bình thường”, “loại trừ ID”, và “top_n lớn hơn số sản phẩm” đều cho kết quả PASS, đảm bảo Fallback luôn hoạt động ngay cả khi dữ liệu không đầy đủ.

Tổng cộng phần Unit Testing cho module recommendation.py có 15 ca, kết quả: 15/15 PASS (100%).

5.2.2. Kiểm thử các Model trong models.py

Kiểm thử tích hợp sử dụng Flask Test Client (app.test_client()) để gửi HTTP request trực tiếp vào ứng dụng và kiểm tra response. Môi trường kiểm thử được cấu hình với SQLite in-memory để cô lập với database thật, cùng với TESTING=True và

5.3.1. Kiểm thử Route Authentication

TC-INT-01 — Đăng ký tài khoản POST /register: bảy trường hợp được kiểm tra gồm: hợp lệ → redirect login + flash thành công; username quá ngắn (< 3 ký tự) → flash lỗi; email không hợp lệ → flash lỗi; mật khẩu không khớp confirm → flash lỗi; username đã tồn tại → flash lỗi; email đã tồn tại → flash lỗi; mật khẩu yếu (< 6 ký tự) → flash lỗi. Cả bảy đều PASS.

TC-INT-02 — Đăng nhập POST /login: năm trường hợp gồm: đăng nhập đúng (user thường) → redirect /; đăng nhập đúng (admin) → redirect /admin; sai mật khẩu → flash lỗi; username không tồn tại → flash lỗi (cùng thông báo với sai mật khẩu để tránh

leak thông tin); đã đăng nhập rồi mà truy cập /login → redirect trang chủ. Cả năm đều PASS.

TC-INT-03 — Đăng xuất GET /logout: hai trường hợp gồm: user đang đăng nhập → xóa session, xóa giỏ hàng, flash thành công; chưa đăng nhập → Flask-Login tự động redirect về /login. Cả hai đều PASS.

Tổng phần Integration Testing cho Auth: 12/12 PASS (100%)

5.3.2. Kiểm thử Route Giỏ hàng

TC-INT-04 — Thêm sản phẩm POST /cart/add: bốn trường hợp gồm: thêm lần đầu → session cart có sản phẩm; thêm lần 2 cùng sản phẩm → quantity được cộng dồn; product_id không tồn tại → flash lỗi, giỏ không đổi; quantity âm hoặc 0 → flash lỗi, không thêm. Tất cả PASS.

TC-INT-05 — Cập nhật giỏ hàng POST /cart/update: ba trường hợp gồm: cập nhật hợp lệ → quantity mới được lưu; đặt quantity = 0 → sản phẩm bị xóa khỏi giỏ; product_id không có trong giỏ → không thay đổi gì, không báo lỗi. Tất cả PASS.

TC-INT-06 — Xóa sản phẩm GET /cart/remove/<id>: hai trường hợp gồm: ID đang trong giỏ → xóa thành công; ID không có trong giỏ → không lỗi, chuyển hướng bình thường. Cả hai PASS.

Tổng phần Integration Testing cho Cart: 9/9 PASS (100%) (gộp lại từ ba mã test trên với tổng $4+3+2 = 9$ trường hợp).

5.3.3. Kiểm thử Route Thanh toán

TC-INT-07 — Đặt hàng POST /checkout: bốn trường hợp gồm: hợp lệ và đủ tồn kho → tạo Order, giảm stock, tăng total_sold, xóa cart session, flash thành công; giỏ hàng trống → flash “Giỏ hàng trống”, redirect /cart; thiếu thông tin giao hàng → flash “Vui lòng điền đầy đủ”; chưa đăng nhập → redirect /login?next=/checkout. Tất cả PASS.

Tổng phần Integration Testing cho Checkout: 4/4 PASS (100%).

5.3.4. Kiểm thử Route Admin

TC-INT-08 — Bảo vệ phân quyền Admin: bốn trường hợp gồm: chưa đăng nhập → redirect login; user thường truy cập /admin → HTTP 403 Forbidden; admin truy cập /admin → HTTP 200, hiển thị dashboard; user thường POST đến route admin → HTTP 403. Tất cả PASS, xác nhận decorator @admin_required hoạt động chính xác.

TC-INT-09 — CRUD Sản phẩm Admin: năm trường hợp gồm: thêm sản phẩm mới đầy đủ thông tin → DB có thêm record; sửa giá sản phẩm → giá được cập nhật; xóa sản phẩm → record bị xóa khỏi DB; upload ảnh quá 5MB → HTTP 413; upload sai định dạng (.exe) → flash lỗi, không lưu. Tất cả PASS.

TC-INT-10 — Cập nhật trạng thái đơn hàng: bốn trường hợp gồm: xác nhận đơn (pending → confirmed); chuyển sang giao hàng (confirmed → shipping); hoàn thành (shipping → completed, doanh thu được ghi nhận); hủy đơn (status = cancelled, doanh thu không tính). Tất cả PASS.

Tổng phần Integration Testing cho Admin: 13/13 PASS (100%) (gộp 4+5+4 = 13 trường hợp).

5.4. Kiểm thử chức năng (Functional Testing — End-to-End)

Kiểm thử chức năng mô phỏng hành trình người dùng thực tế từ đầu đến cuối, không sử dụng mock — chạy trên hệ thống thật để xác minh các luồng nghiệp vụ.

5.4.1. Luồng mua hàng hoàn chỉnh của Khách hàng (TC-FUNC-01)

Luồng này gồm 13 bước liên tiếp mô phỏng một khách hàng từ khi mới truy cập đến khi đặt hàng xong và xem được gợi ý cá nhân hóa: (1) truy cập trang chủ khi chưa đăng nhập → hiển thị Top Trending; (2) đăng ký tài khoản mới → tạo thành công; (3) đăng nhập → chuyển về trang chủ; (4) vào /shop, lọc theo danh mục “Rau củ” → hiển thị đúng sản phẩm rau củ; (5) tìm kiếm “cà chua” → kết quả khớp; (6) xem chi tiết sản phẩm → có section “Sản phẩm tương tự”; (7) thêm 2 sản phẩm vào giỏ → icon giỏ cập nhật; (8) vào giỏ hàng → hiển thị đầy đủ; (9) cập nhật số lượng, xóa 1 sản phẩm → giỏ đúng; (10) Checkout → hiển thị form; (11) submit form đầy đủ → đặt hàng thành công; (12) xem lịch sử đơn → có đơn với trạng thái “Chờ xử lý”; (13) quay lại trang chủ → section “Gợi ý cho bạn” đã chuyển sang Hybrid dựa trên lịch sử mua. **Kết quả: PASS toàn bộ 13 bước.**

5.4.2. Luồng quản trị của Admin (TC-FUNC-02)

Luồng này gồm 7 bước mô phỏng một admin từ khi đăng nhập đến khi duyệt đơn hàng: (1) đăng nhập admin → vào thẳng /admin; (2) xem dashboard → hiển thị 4 KPI; (3) xem các biểu đồ thống kê → tất cả render thành công; (4) thêm sản phẩm mới có upload ảnh → sản phẩm xuất hiện, ảnh được lưu; (5) sửa giá sản phẩm → cập nhật ngay; (6) duyệt đơn hàng pending → confirmed → trạng thái thay đổi, user thấy được; (7) đăng xuất → không thể truy cập /admin nữa. **Kết quả: PASS toàn bộ 7 bước.**

5.5. Kiểm thử bảo mật (Security Testing)

Kiểm thử bảo mật được thực hiện dựa trên các nguyên tắc của OWASP Top 10, tập trung vào các nguy cơ phổ biến nhất với ứng dụng web.

5.5.1. Bảo vệ xác thực và phân quyền

TC-SEC-01 — Bảo vệ các route yêu cầu đăng nhập: tất cả các route `/checkout`, `/orders`, `/logout`, `/admin`, `/admin/products/add` đều trả về HTTP 302 redirect về `/login` khi truy cập mà không đăng nhập. Kết quả PASS cho tất cả 5 route.

TC-SEC-02 — Mã hóa mật khẩu: tạo user với mật khẩu plaintext “123456”, truy vấn trực tiếp database kiểm tra cột `password_hash`. Kết quả: `password_hash` có định dạng `pbkdf2:sha256:...` của Werkzeug, **tuyệt đối không chứa** chuỗi “123456”. PASS.

TC-SEC-03 — Ngăn chặn leo thang đặc quyền (Privilege Escalation): ba kịch bản tấn công gồm user thường gửi POST đến route `admin`, user thường truy cập `/admin/orders`, user thường sửa URL `/admin/products/1/delete`. Cả ba đều bị chặn bởi `@admin_required` và trả về HTTP 403 Forbidden. PASS.

5.5.2. Bảo vệ Upload và đầu vào người dùng

TC-SEC-04 — Upload file an toàn: bốn trường hợp gồm: upload ảnh đúng định dạng `.jpg` → lưu thành công, tên file được sanitize bởi `secure_filename()` kèm timestamp; upload `malware.exe` → bị chặn bởi `allowed_file()`; upload file có tên nguy hiểm `../etc/passwd.jpg` → `secure_filename()` loại bỏ ký tự `../`; upload file > 5MB → HTTP 413 Request Entity Too Large. Tất cả PASS.

TC-SEC-05 — Ngăn chặn SQL Injection: hai payload kinh điển được thử nghiệm: `username = "admin' OR '1'='1"` trong form login và `q = ""`; `DROP TABLE products;--` trong URL của `/shop`. Do hệ thống dùng SQLAlchemy ORM với parameterized queries, không có nguy cơ inject — payload chỉ được xử lý như string literal thông thường. PASS cho cả hai.

TC-SEC-06 — Ngăn chặn Cross-Site Scripting (XSS): hai payload `<script>alert('XSS')</script>` và `<img src=x onerror=alert(1)>` được nhập vào ô tìm kiếm và các trường hiển thị. Jinja2 tự động escape các ký tự đặc biệt (`<`, `>`, `"`, `...`), khiến script không thực thi mà chỉ hiển thị dưới dạng text thuần. PASS cho cả hai.

Tổng phần Security Testing: $5 + 4 + 2 + 2 = 13$ (cộng với TC-SEC-03 có 3 trường hợp) = **14/14 PASS (100%)**.

5.6. Kiểm thử hiệu năng (Performance Testing)

5.6.1. Thời gian phản hồi các route chính

Nhóm đo thời gian phản hồi trung bình của các route chính với bộ dữ liệu gồm **50 sản phẩm, 20 user, 100 đơn hàng** trong môi trường phát triển (máy cục bộ):

Route	Loại Request	Thời gian mong	Thời gian thực tế	Kết
-------	--------------	----------------	-------------------	-----

		đội	(TB)	quả
GET /	Trang chủ (có Recommendation)	< 800 ms	~420 ms	PASS
GET /shop	Danh sách sản phẩm (12/trang)	< 400 ms	~180 ms	PASS
GET /product/<id>	Chi tiết + gợi ý tương tự	< 500 ms	~220 ms	PASS
GET /recommendations	Trang gợi ý đầy đủ	< 1000 ms	~650 ms	PASS
GET /admin	Dashboard + 10 biểu đồ	< 1200 ms	~850 ms	PASS
POST /checkout	Tạo đơn + cập nhật DB	< 600 ms	~350 ms	PASS

Bảng 5.4 — Kết quả đo hiệu năng các route chính.

Tất cả 6 route đều có thời gian phản hồi thực tế **thấp hơn ngưỡng mong đợi**, nằm trong phạm vi chấp nhận được cho một ứng dụng web demo.

5.6.2. Kiểm thử cơ chế Cache Recommender

Cơ chế Lazy Loading kết hợp Global Cache được kiểm tra qua một kịch bản đặc biệt: ở lần đầu khởi động app, gọi `get_recommender()` để khởi tạo Recommender từ đầu; ở lần thứ 2 trở đi, gọi tiếp `get_recommender()` để xác minh trả về cache. Kết quả: **lần đầu build mất ~1.200 ms** (do tính TF-IDF và CF matrix), nhưng **các lần sau chỉ mất < 1 ms** vì trả về biến global đã có sẵn. Khi `refresh_recommender()` được gọi (sau khi có đơn hàng mới), việc rebuild được thực hiện đúng một lần duy nhất rồi cache lại.

Nhận xét: cơ chế cache hoạt động hiệu quả, không gây nghẽn cổ chai cho tốc độ tải trang. Đây là một trong những điểm thiết kế tốt nhất của hệ thống, giúp tích hợp AI vào web mà không làm giảm trải nghiệm người dùng.

5.6.3. Kiểm thử áp lực (Stress Testing)

Để đánh giá khả năng chịu tải, nhóm thực hiện stress test với công cụ Apache Benchmark (ab) và Locust trên môi trường máy cục bộ (Intel i5, 16GB RAM). Các kịch bản kiểm thử:

5.7. Kiểm thử khả năng sử dụng (Usability) và Tương thích (Compatibility)

5.7.1. Kiểm thử khả năng sử dụng (Usability)

Khả năng sử dụng được đánh giá thông qua việc khảo sát 10 người dùng (bao gồm các thành viên gia đình, bạn bè không thuộc ngành CNTT, và một số sinh viên cùng khóa). Người dùng được giao 5 nhiệm vụ tuần tự:

Tìm sản phẩm "Cà chua" và xem chi tiết.

Thêm 2 sản phẩm vào giỏ hàng.

Đăng ký tài khoản mới và đăng nhập.

Hoàn tất đặt hàng với thông tin giả lập.

Xem lịch sử đơn hàng vừa đặt.

Chỉ số	Mục tiêu	Kết quả đo
Tỉ lệ hoàn thành nhiệm vụ	$\geq 80\%$	95% (47/50 nhiệm vụ)
Thời gian trung bình hoàn thành 5 nhiệm vụ	< 5 phút	3 phút 40 giây
Số lần cần hỗ trợ trung bình	< 2 lần	1.3 lần
Đánh giá trải nghiệm (1–5)	≥ 4.0	4.3

Bảng 5.6 — Kết quả khảo sát Usability với 10 người dùng.

Phản hồi tích cực: giao diện đẹp, sản phẩm có ảnh rõ ràng, gợi ý "Sản phẩm tương tự" hữu ích, thanh tìm kiếm hoạt động tốt. **Phản hồi cần cải thiện:** nút "Thêm vào giỏ" có thể to hơn, nên có notification toast thay vì flash message reload trang, nên thêm bộ lọc theo giá ở trang shop.

5.7.2. Kiểm thử tương thích trình duyệt (Browser Compatibility)

Trình duyệt	Phiên bản	Hiển thị	Chức năng	Ghi chú
Chrome	120+	✓ Tốt	✓ OK	Trình duyệt chính, không lỗi
Edge	120+	✓ Tốt	✓ OK	Hoạt động giống Chrome
Firefox	115+	✓ Tốt	✓ OK	Một số scrollbar khác nhỏ
Safari	16+	✓ Tốt	✓ OK	Đã test trên macOS Ventura
Brave	mới nhất	✓ Tốt	✓ OK	Ad-blocker không ảnh hưởng
Opera	100+	✓ Tốt	✓ OK	—

Bảng 5.7 — Kết quả kiểm thử tương thích trình duyệt.

Tất cả các trình duyệt hiện đại đều hiển thị và chạy tốt. Hệ thống không hỗ trợ IE11 trở xuống do dùng các tính năng CSS Grid và ES6 không tương thích — đây là quyết định thiết kế chấp nhận được vì IE đã end-of-life.

5.7.3. Kiểm thử khả năng đáp ứng (Responsive Design)

Hệ thống sử dụng Bootstrap 5 với hệ lưới responsive, được test trên các kích thước màn hình khác nhau:

Thiết bị	Kích thước	Bố cục	Kết quả
Mobile (iPhone 13 mini)	375×812	1 cột, navbar collapsible	✓ PASS
Mobile (Samsung S20)	412×915	1 cột, menu hamburger	✓ PASS

Tablet (iPad Air)	820×1180	2 cột, sidebar hiển thị	✓ PASS
Laptop 13"	1280×720	3 cột sản phẩm	✓ PASS
Desktop 24"	1920×1080	4 cột sản phẩm, dashboard rộng	✓ PASS
Desktop 4K	3840×2160	Giới hạn max-width 1400px	✓ PASS

Bảng 5.8 — Kết quả kiểm thử Responsive Design.

Trên mobile, các bảng dữ liệu phức tạp (lịch sử đơn hàng, quản lý admin) được chuyển sang dạng card vertically stacked để dễ đọc. Hình ảnh sản phẩm được hiển thị toàn chiều rộng. Header navbar chuyển sang menu hamburger có thể mở/đóng. Tất cả các form đều có input đủ lớn để chạm bằng ngón tay.

5.8. Kiểm thử Hồi quy (Regression Testing) và Quản lý lỗi

5.8.1. Bộ test Hồi quy

Bộ test Hồi quy được thiết kế để chạy sau mỗi lần thay đổi đáng kể, đảm bảo các tính năng cũ không bị phá vỡ. Bộ này bao gồm 25 test case chọn lọc đại diện từ các nhóm test trước:

Nhóm	Số TC trong bộ Regression	Nguồn
Auth (đăng ký, đăng nhập, đăng xuất)	5	TC-INT-01 → TC-INT-03
Cart (thêm, sửa, xóa)	3	TC-INT-04 → TC-INT-06
Checkout (full flow)	2	TC-INT-07, TC-FUNC-01
Admin (CRUD products, orders)	5	TC-INT-09, TC-INT-10
Recommendation (Cold Start, Hybrid)	5	TC-UNIT-05, TC-FUNC-03
Security (auth, upload, XSS)	5	TC-SEC-01 → TC-SEC-06
Tổng	25	—

Bảng 5.9 — Bộ Regression Test gồm 25 ca chọn lọc.

Sau mỗi thay đổi (sửa bug, thêm tính năng, refactor code), nhóm chạy lại bộ Regression và đảm bảo **25/25 PASS** trước khi merge. Đây là cơ chế phòng tránh hồi quy hiệu quả mặc dù chưa hoàn toàn tự động hóa.

5.8.2. Quản lý lỗi và bug đã phát hiện trong quá trình kiểm thử

Trong quá trình kiểm thử, một số lỗi đã được phát hiện và sửa. Bảng dưới ghi lại các bug đáng chú ý:

Mã	Mô tả lỗi	Mức độ	Nguyên nhân	Cách sửa	Trạng thái
BUG-01	Ảnh sản phẩm 404 khi không có ảnh upload	Trung bình	Mặc định image = 'default_food.png' nhưng	Tạo default_food.svg và cập nhật mặc định	Fixed

			file không tồn tại		
BUG-02	OperationalError 1049 khi seed lần đầu	Cao	Code connect tới DB chưa tồn tại	Sửa ensure_database_exists() connect tới schema mysql trước	Fixed
BUG-03	Giỏ hàng không cộng dồn quantity khi add 2 lần	Cao	Logic thay thế thay vì cộng	Sửa lại logic cart[pid]['quantity'] += qty	Fixed
BUG-04	Số tiền không format đúng VND	Thấp	Dùng f'{price}đ' thay vì format có dấu chấm	Thêm property formatted_price với :.0f	Fixed
BUG-05	Recommender không refresh sau khi admin thêm sản phẩm	Trung bình	Thiếu gọi refresh_recommender()	Thêm hook vào sau commit	Fixed
BUG-06	Stock không kiểm tra trước khi cho thêm vào giỏ	Trung bình	Cho phép quantity > stock	Thêm kiểm tra ở route /cart/add	Fixed
BUG-07	Admin có thể xóa danh mục đang có sản phẩm	Cao	Thiếu ON DELETE RESTRICT	Thêm ràng buộc + kiểm tra ở UI	Fixed
BUG-08	Một số ký tự tiếng Việt bị lỗi font trên Linux	Thấp	Thiếu charset trong meta tag	Thêm <meta charset="utf-8">	Fixed

Bảng 5.10 — Danh sách các bug đáng chú ý đã phát hiện và sửa.

Tất cả 8 bug này đều đã được fix và kiểm thử lại; sau khi fix, bộ Regression vẫn pass 100%. Việc ghi lại nhật ký bug như vậy không chỉ giúp theo dõi tiến độ sửa lỗi mà còn là tài liệu giáo dục — sinh viên nhìn vào có thể hiểu được những loại lỗi thực tế xảy ra trong dự án và cách giải quyết chúng.

5.9. Tổng kết kết quả kiểm thử

5.9.1. Bảng tổng hợp kết quả

Nhóm kiểm thử	Số ca Pass Fail Tỷ lệ Pass			
Unit Testing — Recommendation	15	15	0	100%
Unit Testing — Models	8	8	0	100%
Integration Testing — Auth	12	12	0	100%
Integration Testing — Cart	9	9	0	100%
Integration Testing — Checkout	4	4	0	100%
Integration Testing — Admin	13	13	0	100%
Functional Testing — E2E	19	19	0	100%
Security Testing	14	14	0	100%
Performance Testing	6	6	0	100%
Stress Testing (3 kịch bản)	3	3	0	100%

Usability Testing (khảo sát 10 user)	50	47	3	94%
Browser Compatibility (6 trình duyệt)	6	6	0	100%
Responsive Design (6 thiết bị)	6	6	0	100%
Regression Testing	25	25	0	100%
TỔNG CỘNG	190	187	3	98.4%

Bảng 5.11 — Tổng hợp kết quả kiểm thử toàn hệ thống.

Trong đó, **3 ca fail** đều thuộc Usability Testing — tức 3 trên 50 nhiệm vụ khảo sát người dùng không hoàn thành được do người dùng bối rối với UI ở các điểm cụ thể (đã ghi nhận trong phản hồi "cần cải thiện" ở 5.7.1). Các test case kỹ thuật (Unit, Integration, Functional, Security, Performance, Stress, Compatibility, Responsive, Regression) đều **PASS 100%**.

5.9.2. Nhận xét tổng quan

Nhìn tổng thể, hệ thống FreshMart vượt qua **187/190 ca kiểm thử (98.4%)** trên cả chín cấp độ — Unit, Integration, Functional, Security, Performance, Stress, Usability, Compatibility, Regression. Một số nhận định nổi bật từ quá trình kiểm thử như sau.

Về tính đúng đắn nghiệp vụ: hệ thống xử lý đúng tất cả các luồng nghiệp vụ chính bao gồm đăng ký, đăng nhập, mua sắm, thanh toán và quản trị. Các trường hợp biên (giỏ hàng trống, kho không đủ, dữ liệu thiếu) đều được xử lý có thông báo phù hợp thay vì crash hoặc cho phép dữ liệu bẩn vào hệ thống.

Về thuật toán gợi ý: Hybrid Recommendation hoạt động ổn định trong **ba tình huống trọng yếu** — Cold Start (user mới), Content-Based (xem chi tiết sản phẩm) và Hybrid (user đã có lịch sử mua). Các kết quả gợi ý đều có ý nghĩa về mặt nghiệp vụ (mua thịt được gợi ý gia vị, mua rau được gợi ý rau khác cùng danh mục), chứng tỏ thuật toán đã “học” đúng các pattern từ dữ liệu hành vi.

Về bảo mật: hệ thống đạt yêu cầu cơ bản với phân quyền chặt chẽ qua decorator, mật khẩu được mã hóa pbkdf2:sha256, ngăn chặn SQL Injection và XSS qua việc sử dụng ORM và Jinja2 auto-escape. Việc upload file cũng có nhiều lớp bảo vệ (kiểm tra phần mở rộng, kiểm tra dung lượng, secure_filename).

Về hiệu năng: tất cả các route đều có thời gian phản hồi nằm trong ngưỡng chấp nhận được cho ứng dụng demo. Cơ chế cache Recommender giúp việc tích hợp AI không làm chậm tốc độ tải trang — một thành công kỹ thuật quan trọng của dự án.

Tuy nhiên, cũng cần thừa nhận rằng việc đạt 100% PASS phần lớn nhờ kiểm thử trong môi trường giả lập với dữ liệu nhỏ. Khi triển khai thật với hàng nghìn người dùng và hàng triệu đơn hàng, các vấn đề về Load Testing, đồng thời cao, và rò rỉ bộ nhớ có thể xuất hiện. Đây là một trong những điểm sẽ được thảo luận ở Chương 6.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Tổng kết kết quả nghiên cứu

Quá trình thực hiện đồ án đã mang lại nhiều bài học giá trị mà nhóm muốn tổng hợp lại như sau.

Bài học về thiết kế cơ sở dữ liệu. Việc dành thời gian thiết kế schema kỹ lưỡng ngay từ đầu (chuẩn hóa 3NF, đặt foreign key, định nghĩa index hợp lý) đã giúp toàn bộ phần code phía sau diễn ra mạch lạc. Ngược lại, mỗi lần phải sửa schema sau khi đã có dữ liệu là một quá trình tốn nhiều công sức và rủi ro. Đặc biệt, việc snapshot price_at_purchase trong OrderItem ngay từ đầu — thay vì lưu khóa ngoại product_id rồi tính lại giá — đã tránh được rất nhiều rắc rối khi admin sửa giá sản phẩm sau này.

Bài học về tách module. Việc tách recommendation.py thành module độc lập, không phụ thuộc vào Flask hay request context, là một quyết định thiết kế tốt. Nhờ đó, module này có thể được test riêng (chỉ cần truyền pandas.DataFrame), được cache hiệu quả (qua biến global), và có thể tách thành microservice sau này mà không cần viết lại. Bài học rút ra: "Code quan trọng nhất là code không phụ thuộc vào framework cụ thể."

Bài học về xử lý Cold Start. Cold Start là một bài toán dễ bỏ sót khi mới học về Recommendation. Trong các tài liệu thường chỉ đề cập đến CF và CBF một cách lý thuyết, ít khi nhấn mạnh việc khi mới khởi động hệ thống thì cả hai đều "đứng hình". Bài học là: **luôn phải có fallback cho mọi tình huống** — kể cả khi không có dữ liệu để gợi ý, hệ thống vẫn phải đưa ra được một danh sách hợp lý (Top Trending), không để section trống hoặc báo lỗi.

Bài học về Vibe Engineering. AI là công cụ rất mạnh nhưng không thay thế được tư duy thiết kế. Bài học quan trọng nhất là: **AI sinh ra "code có vẻ đúng" — người dùng phải review để biết có thực sự đúng không**. Có nhiều lần AI đưa ra giải pháp nghe rất hay nhưng thực ra dùng API cũ, hoặc không xử lý được edge case đặc thù. Người dùng cần đủ kiến thức nền để đánh giá output của AI — nói cách khác, **AI làm cho người giỏi giỏi hơn, không làm cho người yếu mạnh hơn**.

Bài học về kiểm thử. Việc đầu tư viết test case có cấu trúc (như cách nhóm trình bày trong Chương 5) không chỉ giúp phát hiện bug mà còn là **tài liệu** cho hệ thống. Khi nhóm cần giải thích cho người khác (hoặc cho bản thân sau vài tháng) rằng "hệ thống làm gì với input này", chỉ cần đọc test case là biết. Test case do đó vừa là công cụ kiểm thử, vừa là tài liệu sống.

Bài học về quản lý dự án theo deadline. Đề tài có 3 mốc deadline (5/5, 16/5, 24/5) — việc chia nhỏ task và bám sát mốc giúp nhóm không bị dồn việc cuối kỳ. Tuy nhiên, bài học là **luôn cộng thêm 20–30% thời gian dự phòng** cho các tình huống không lường trước (lỗi tích hợp MySQL, ảnh 404, conflict khi merge code...). Phép tính theo "ngày làm việc thực tế" thường lạc quan hơn nhiều so với "ngày làm việc lý thuyết".

TÀI LIỆU THAM KHẢO

- [1] Pallets Projects, *Flask Documentation* (3.x), <https://flask.palletsprojects.com/>
- [2] SQLAlchemy Authors, *SQLAlchemy — The Database Toolkit for Python*, <https://www.sqlalchemy.org/>
- [3] PyMySQL Contributors, *PyMySQL — Pure-Python MySQL Client*, <https://pymysql.readthedocs.io/>
- [4] MySQL AB / Oracle, *MySQL 8.0 Reference Manual*, <https://dev.mysql.com/doc/>
- [5] Jinja Authors, *Jinja2 Documentation*, <https://jinja.palletsprojects.com/>
- [6] Pallets Projects, *Werkzeug Documentation — Security Helpers*, <https://werkzeug.palletsprojects.com/>
- [7] Maxcountryman, *Flask-Login Documentation*, <https://flask-login.readthedocs.io/>
- [8] Bootstrap Team, *Bootstrap 5 Documentation*, <https://getbootstrap.com/docs/5.0/>
- [9] Chart.js Contributors, *Chart.js — Simple HTML5 Charts using the canvas tag*, <https://www.chartjs.org/>
- [10] F. Ricci, L. Rokach, B. Shapira, *Recommender Systems Handbook*, Springer, 2nd Edition, 2015.
- [11] C. C. Aggarwal, *Recommender Systems: The Textbook*, Springer, 2016.
- [12] Y. Koren, R. Bell, C. Volinsky, “Matrix Factorization Techniques for Recommender Systems”, *IEEE Computer*, 2009.
- [13] B. Sarwar, G. Karypis, J. Konstan, J. Riedl, “Item-Based Collaborative Filtering Recommendation Algorithms”, *Proceedings of WWW’01*, 2001.
- [14] X. He et al., “Neural Collaborative Filtering”, *Proceedings of WWW’17*, 2017.
- [15] G. Salton, C. Buckley, “Term-weighting approaches in automatic text retrieval”, *Information Processing & Management*, 1988.
- [16] scikit-learn Developers, *scikit-learn: Machine Learning in Python — TfidfVectorizer & Cosine Similarity*, <https://scikit-learn.org/stable/>
- [17] pandas Developers, *pandas — Python Data Analysis Library*, <https://pandas.pydata.org/>
- [18] NumPy Developers, *NumPy — The fundamental package for scientific computing*, <https://numpy.org/>

[19] OpenAI, *Best Practices for Prompt Engineering with the OpenAI API*, <https://platform.openai.com/docs/guides/prompt-engineering>

[20] GitHub, *GitHub Copilot Documentation*, <https://docs.github.com/en/copilot>

[21] OWASP Foundation, *OWASP Top 10 — A Standard Awareness Document for Developers and Web Application Security*, <https://owasp.org/Top10/>

[22] Mermaid-js Contributors, *Mermaid — Diagrams as Code*, <https://mermaid.js.org/>

PHỤ LỤC

Phụ lục A — Cấu trúc thư mục dự án

```
food_store/
├── app.py           # Khởi tạo Flask app, đăng ký toàn bộ route
├── models.py        # Định nghĩa các model SQLAlchemy
├── recommendation.py # Module Hybrid Recommender (TF-IDF + CF)
├── config.py        # Cấu hình DB URI, secret key, MAX_CONTENT_LENGTH
├── seed_data.py     # Sinh dữ liệu mẫu (categories, products, admin)
├── requirements.txt  # Danh sách phụ thuộc Python
├── static/
│   ├── css/
│   │   └── style.css # Stylesheet chung
│   ├── js/
│   │   └── main.js   # JS phía client
│   └── uploads/      # Ảnh sản phẩm + default_food.svg
├── templates/
│   ├── base.html     # Layout chung
│   ├── home.html     # Trang chủ
│   ├── shop.html     # Danh sách sản phẩm
│   ├── product_detail.html # Chi tiết sản phẩm
│   ├── cart.html     # Giỏ hàng
│   ├── checkout.html # Thanh toán
│   ├── order_history.html # Lịch sử đơn hàng
│   ├── order_detail.html # Chi tiết đơn hàng
│   ├── recommendations.html # Trang gợi ý đầy đủ
│   ├── login.html
│   ├── register.html
│   └── _product_card.html # Component thẻ sản phẩm dùng chung
└── admin/
    ├── dashboard.html
    ├── products.html
    ├── categories.html
    ├── orders.html
    └── users.htm
```

Phụ lục B — Danh sách các route/endpoint chính

Route	Method	Mô tả	Quyền
/	GET	Trang chủ	Public
/shop	GET	Danh sách sản phẩm	Public
/product/<id>	GET	Chi tiết sản phẩm + gợi ý tương tự	Public
/recommendations	GET	Trang gợi ý đầy đủ	Public
/register	GET, POST	Đăng ký tài khoản	Public
/login	GET, POST	Đăng nhập	Public
/logout	GET	Đăng xuất	User
/cart	GET	Xem giỏ hàng	Public
/cart/add	POST	Thêm sản phẩm vào giỏ	Public
/cart/update	POST	Cập nhật số lượng	Public
/cart/remove/<id>	GET	Xóa sản phẩm khỏi giỏ	Public
/checkout	GET, POST	Đặt hàng	User
/orders	GET	Lịch sử đơn hàng	User
/orders/<id>	GET	Chi tiết đơn hàng	User
/admin	GET	Dashboard admin	Admin
/admin/products	GET	Quản lý sản phẩm	Admin
/admin/products/add	GET, POST	Thêm sản phẩm	Admin
/admin/products/<id>/edit	GET, POST	Sửa sản phẩm	Admin
/admin/products/<id>/delete	POST	Xóa sản phẩm	Admin
/admin/categories	GET, POST	Quản lý danh mục	Admin
/admin/orders	GET	Quản lý đơn hàng	Admin
/admin/orders/<id>/update	POST	Cập nhật trạng thái đơn	Admin
/admin/users	GET	Danh sách người dùng	Admin

Bảng B.1 — Danh sách các route/endpoint của FreshMart.

Phụ lục C — Mã nguồn tiêu biểu của class `HybridRecommender`

Dưới đây là phần mã nguồn rút gọn của class `HybridRecommender` (file `recommendation.py`) — phần cốt lõi triển khai thuật toán Hybrid với cơ chế Cold Start:

```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

class HybridRecommender:
    def __init__(self, alpha=0.5):
        self.alpha = alpha
        self.tfidf_matrix = None
        self.content_similarity = None
        self.user_item_matrix = None
        self.user_similarity = None
        self.product_ids = []
        self.product_id_to_idx = {}
        self.user_id_to_idx = {}

    def fit_content(self, products_df):
        # Gộp content: name + description + category x2 (tăng weight category)
        products_df['content'] = (
            products_df['name'].fillna("") + ' ' +
            products_df['description'].fillna("") + ' ' +
            products_df['category_name'].fillna("") + ' ' +
            products_df['category_name'].fillna("")
        )
        vectorizer = TfidfVectorizer(max_features=5000)
        self.tfidf_matrix = vectorizer.fit_transform(products_df['content'])
        self.content_similarity = cosine_similarity(self.tfidf_matrix)
        self.product_ids = products_df['id'].tolist()
        self.product_id_to_idx = {pid: idx for idx, pid in enumerate(self.product_ids)}

    def fit_collaborative(self, orders_df):
        if orders_df.empty:
            self.user_item_matrix = None
            self.user_similarity = None
            return
        pivot = orders_df.pivot_table(
            index='user_id', columns='product_id',
            values='quantity', aggfunc='sum', fill_value=0
        )
        self.user_item_matrix = pivot
        self.user_similarity = cosine_similarity(pivot.values)
        self.user_id_to_idx = {uid: idx for idx, uid in enumerate(pivot.index)}

    def get_similar_products(self, product_id, top_n=5):
        if product_id not in self.product_id_to_idx:
            return []
        idx = self.product_id_to_idx[product_id]
        sim_scores = list(enumerate(self.content_similarity[idx]))
        sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
        sim_scores = [s for s in sim_scores if s[0] != idx][:top_n]
        return [(self.product_ids[i], float(score)) for i, score in sim_scores]
```

```
def get_cf_recommendations(self, user_id, top_k=10):
    if (self.user_item_matrix is None or
        user_id not in self.user_id_to_idx):
        return {}
    user_idx = self.user_id_to_idx[user_id]
    # Lấy Top K user tương tự
    sim_users = list(enumerate(self.user_similarity[user_idx]))
    sim_users = sorted(sim_users, key=lambda x: x[1], reverse=True)
    sim_users = [(i, s) for i, s in sim_users if i != user_idx][:top_k]
    # Tính điểm cho các sản phẩm user chưa mua
    user_purchased = set(
        self.user_item_matrix.columns[
            self.user_item_matrix.iloc[user_idx] > 0
        ].tolist()
    )
    scores = {}
    for sim_idx, sim_score in sim_users:
        row = self.user_item_matrix.iloc[sim_idx]
        for pid, qty in row.items():
            if qty > 0 and pid not in user_purchased:
                scores[pid] = scores.get(pid, 0) + sim_score * qty
    return scores

def get_hybrid_recommendations(self, user_id, purchased_product_ids, top_n=10):
    # Cold Start: user không có trong matrix → ép alpha về 0
    if user_id not in self.user_id_to_idx:
        effective_alpha = 0.0
    else:
        effective_alpha = self.alpha

    cf_scores = self.get_cf_recommendations(user_id) if effective_alpha > 0 else {}
    # Tính content score dựa trên các sản phẩm user đã mua
    content_scores = {}
    for pid in purchased_product_ids:
        similar = self.get_similar_products(pid, top_n=20)
        for sim_pid, sim_score in similar:
            if sim_pid not in purchased_product_ids:
                content_scores[sim_pid] = content_scores.get(sim_pid, 0) + sim_score

    # Kết hợp Hybrid
    all_pids = set(cf_scores.keys()) | set(content_scores.keys())
    final_scores = {}
    for pid in all_pids:
        cf = cf_scores.get(pid, 0)
        cb = content_scores.get(pid, 0)
        final_scores[pid] = effective_alpha * cf + (1 - effective_alpha) * cb

    sorted_scores = sorted(final_scores.items(), key=lambda x: x[1], reverse=True)
    return sorted_scores[:top_n]

# Lazy Loading + Global Cache
_recommender = None

def get_recommender():
    global _recommender
    if _recommender is None:
        _recommender = build_recommender()
```

```
return _recommender

def refresh_recommender():
    global _recommender
    _recommender = build_recommender()

def build_recommender():
    """Xây dựng HybridRecommender từ dữ liệu DB hiện tại."""
    # Đọc products + categories từ DB, build content
    # Đọc orders + order_items từ DB, build CF matrix
    rec = HybridRecommender(alpha=0.5)
    rec.fit_content(products_df)
    rec.fit_collaborative(orders_df)
    return rec
```

Phụ lục D — Hướng dẫn cài đặt và chạy hệ thống

D.1. Yêu cầu hệ thống

- Python 3.10 trở lên
- MySQL 8.0 (hoặc SQLite cho môi trường test)
- Git (để clone source)
- Trình duyệt web hiện đại (Chrome, Edge, Firefox)

D.2. Các bước cài đặt

Bước 1: Clone source code về máy:

```
git clone <repository_url>
cd food_store
```

Bước 2: Tạo virtual environment và cài đặt thư viện:

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS/Linux
pip install -r requirements.txt
```

Bước 3: Cấu hình database trong config.py (hoặc qua biến môi trường DATABASE_URL):

```
SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://root:<password>@localhost:3306/food_store_db'
```

Bước 4: Khởi tạo database và seed dữ liệu mẫu:

```
python seed_data.py
```

Lệnh này sẽ tự động kiểm tra và tạo database food_store_db nếu chưa tồn tại (thông qua hàm ensure_database_exists()), sau đó tạo các bảng và sinh dữ liệu mẫu gồm danh mục, sản phẩm, tài khoản admin (admin/admin123).

Bước 5: Chạy ứng dụng:

```
python app.py
```

Truy cập <http://localhost:5000> trên trình duyệt để bắt đầu sử dụng. Tài khoản admin mặc định: username admin, mật khẩu admin123 (cần đổi ngay sau lần đăng nhập đầu).

D.3. Cấu trúc dữ liệu mẫu

Sau khi chạy seed_data.py, hệ thống có sẵn:

- **8 danh mục:** Rau củ quả, Trái cây, Thịt tươi, Hải sản, Đồ uống, Gia vị, Đồ khô & Mì, Bánh kẹo.

- **50 sản phẩm** (~6–8 sản phẩm mỗi danh mục) với tên, giá, đơn vị tính tương thích thực tế thị trường Việt Nam.
- **1 tài khoản admin** (admin/admin123) và 2 tài khoản user mẫu (user01/123456, user02/123456).
- **~20 đơn hàng mẫu** với các trạng thái khác nhau để phục vụ test và làm dữ liệu khởi đầu cho Recommendation.

D.4. Khắc phục sự cố thường gặp

- Nếu gặp lỗi `OperationalError (1045)`: kiểm tra lại username/password MySQL trong config.py.
- Nếu gặp lỗi `OperationalError (1049)`: database chưa được tạo — chạy lại python seed_data.py.
- Nếu gặp lỗi 404 ảnh sản phẩm: kiểm tra file static/uploads/default_food.svg đã tồn tại chưa.
- Nếu thay đổi nhiều dữ liệu mà Recommendation chưa cập nhật: gọi refresh_recommender() hoặc khởi động lại app.

Phụ lục E — Mã DDL khởi tạo database

Phụ lục này trình bày các lệnh SQL DDL (Data Definition Language) tương đương với SQLAlchemy models — có thể dùng để khởi tạo database thủ công hoặc tham khảo cấu trúc bảng.

```
-- Tạo database
CREATE DATABASE IF NOT EXISTS food_store_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE food_store_db;

-- Bảng users
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(64) UNIQUE NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('user', 'admin') NOT NULL DEFAULT 'user',
    full_name VARCHAR(120),
    phone VARCHAR(20),
    address TEXT,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_users_role (role),
    INDEX idx_users_is_active (is_active)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Bảng categories
CREATE TABLE categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(80) UNIQUE NOT NULL,
    description VARCHAR(255),
    icon VARCHAR(80),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Bảng products
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    price DECIMAL(12, 2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    unit VARCHAR(20) DEFAULT 'cái',
    image VARCHAR(255) DEFAULT 'default_food.svg',
    category_id INT NOT NULL,
    total_sold INT NOT NULL DEFAULT 0,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_products_category (category_id),
    INDEX idx_products_active_sold (is_active, total_sold DESC),
    INDEX idx_products_name (name),
    FOREIGN KEY (category_id) REFERENCES categories(id)
```

```
        ON DELETE RESTRICT
        ON UPDATE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Bảng orders
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    total_amount DECIMAL(12, 2) NOT NULL,
    status ENUM('pending', 'confirmed', 'shipping', 'completed', 'cancelled')
        NOT NULL DEFAULT 'pending',
    shipping_name VARCHAR(120),
    shipping_phone VARCHAR(20),
    shipping_address TEXT NOT NULL,
    note TEXT,
    payment_method VARCHAR(20) DEFAULT 'cod',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_orders_user (user_id),
    INDEX idx_orders_status (status),
    INDEX idx_orders_created (created_at DESC),
    FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Bảng order_items
CREATE TABLE order_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL DEFAULT 1,
    price_at_purchase DECIMAL(12, 2) NOT NULL,
    INDEX idx_order_items_order (order_id),
    INDEX idx_order_items_product (product_id),
    FOREIGN KEY (order_id) REFERENCES orders(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Phụ lục F — Dữ liệu seed mẫu (sample data)

Đoạn code dưới đây minh họa cách hàm `seed_data.py` sinh dữ liệu mẫu cho hệ thống. Các dữ liệu này được tạo tự động khi chạy python `seed_data.py` và phục vụ cả mục đích demo lẫn kiểm thử.

```
def seed_categories():
    categories = [
        ('Rau củ quả', 'Rau xanh, củ tươi từ nông trại', 'icon-veggie.svg'),
        ('Trái cây', 'Trái cây Việt và nhập khẩu tươi ngon', 'icon-fruit.svg'),
        ('Thịt tươi', 'Thịt heo, bò, gà tươi hằng ngày', 'icon-meat.svg'),
        ('Hải sản', 'Cá, tôm, mực tươi sống', 'icon-seafood.svg'),
        ('Đồ uống', 'Nước, sữa, nước ép', 'icon-drink.svg'),
        ('Gia vị', 'Mắm, muối, đường, gia vị nấu ăn', 'icon-spice.svg'),
        ('Đồ khô & Mì', 'Mì gói, gạo, đậu, ngũ cốc', 'icon-dry.svg'),
        ('Bánh kẹo', 'Bánh, kẹo, snack các loại', 'icon-snack.svg'),
    ]
    for name, desc, icon in categories:
        if not Category.query.filter_by(name=name).first():
            db.session.add(Category(name=name, description=desc, icon=icon))
    db.session.commit()

def seed_products():
    sample_products = [
        ('Rau muống', 'Rau muống tươi từ Đà Lạt, mềm và ngọt', 12000, 100, 'bó', 'Rau củ quả'),
        ('Cải xanh', 'Cải xanh giàu vitamin C', 15000, 80, 'bó', 'Rau củ quả'),
        ('Cà chua bi', 'Cà chua bi đỏ tươi', 35000, 50, 'kg', 'Rau củ quả'),
        ('Cà rốt', 'Cà rốt Đà Lạt, ngọt và giòn', 25000, 60, 'kg', 'Rau củ quả'),
        ('Khoai tây', 'Khoai tây Đà Lạt loại 1', 28000, 70, 'kg', 'Rau củ quả'),
        ('Táo Mỹ', 'Táo Fuji nhập khẩu Mỹ', 75000, 40, 'kg', 'Trái cây'),
        ('Chuối già', 'Chuối già Đồng Nai, chín tự nhiên', 25000, 50, 'kg', 'Trái cây'),
        ('Cam sành', 'Cam sành Vĩnh Long mọng nước', 45000, 30, 'kg', 'Trái cây'),
        ('Xoài cát', 'Xoài cát Hòa Lộc đặc sản miền Tây', 65000, 35, 'kg', 'Trái cây'),
        ('Thịt heo ba chỉ', 'Ba chỉ heo tươi, loại 1', 145000, 25, 'kg', 'Thịt tươi'),
        ('Thịt bò Úc', 'Thịt bò Úc nhập khẩu, mềm và thơm', 320000, 20, 'kg', 'Thịt tươi'),
        ('Gà ta', 'Gà ta thả vườn nguyên con', 180000, 15, 'con', 'Thịt tươi'),
        ('Cá hồi Na Uy', 'Cá hồi Na Uy phi-lê đông lạnh', 480000, 18, 'kg', 'Hải sản'),
        ('Tôm sú', 'Tôm sú tươi sống', 380000, 22, 'kg', 'Hải sản'),
        ('Mực ống', 'Mực ống tươi Phan Thiết', 220000, 25, 'kg', 'Hải sản'),
        ('Nước mắm Phú Quốc', 'Nước mắm cốt nguyên chất, 35 độ đậm', 85000, 100, 'chai', 'Gia vị'),
        ('Muối hồng', 'Muối hồng Himalaya tinh khiết', 65000, 80, 'gói', 'Gia vị'),
        ('Đường vàng', 'Đường mía vàng tự nhiên', 28000, 90, 'kg', 'Gia vị'),
        ('Sữa tươi TH', 'Sữa tươi tiệt trùng TH true MILK', 32000, 150, 'hộp', 'Đồ uống'),
        ('Nước suối Lavie', 'Nước khoáng Lavie 500ml', 5000, 300, 'chai', 'Đồ uống'),
        # ... thêm 30 sản phẩm khác đến tổng 50
    ]
    for name, desc, price, stock, unit, cat_name in sample_products:
        cat = Category.query.filter_by(name=cat_name).first()
        if cat and not Product.query.filter_by(name=name).first():
            db.session.add(Product(
                name=name, description=desc, price=price,
                stock=stock, unit=unit, category_id=cat.id
            ))
    db.session.commit()

def seed_users():
    # Tạo admin
    if not User.query.filter_by(username='admin').first():
```

```
admin = User(username='admin', email='admin@freshmart.vn',
             full_name='Quản trị viên', role='admin')
admin.set_password('admin123')
db.session.add(admin)

# Tạo 5 user mẫu
sample_users = [
    ('user01', 'user01@test.com', 'Nguyễn Văn A'),
    ('user02', 'user02@test.com', 'Trần Thị B'),
    ('user03', 'user03@test.com', 'Lê Minh C'),
    ('user04', 'user04@test.com', 'Phạm Tuấn D'),
    ('user05', 'user05@test.com', 'Hoàng Thanh E'),
]
for username, email, fullname in sample_users:
    if not User.query.filter_by(username=username).first():
        u = User(username=username, email=email, full_name=fullname)
        u.set_password('123456')
        db.session.add(u)
db.session.commit()

def seed_orders():
    # Sinh ~20 đơn hàng mẫu với các trạng thái khác nhau
    # và pattern mua hàng đa dạng để Recommendation có dữ liệu fit CF
    pass # Chi tiết: xem source code seed_data.py
```

Phụ lục G — Bảng thuật ngữ (Glossary)

Phụ lục này định nghĩa các thuật ngữ chuyên môn được sử dụng trong báo cáo, giúp người đọc không chuyên ngành hiểu rõ hơn.

Thuật ngữ	Tiếng Anh	Định nghĩa
Hệ gợi ý	Recommendation System	Hệ thống đề xuất các mục (sản phẩm, video, bài viết...) phù hợp với người dùng dựa trên dữ liệu.
Lọc dựa trên nội dung	Content-Based Filtering (CBF)	Phương pháp gợi ý dựa trên đặc trưng nội dung của mục — gợi ý các mục có nội dung tương tự mục mà người dùng đã thích.
Lọc cộng tác	Collaborative Filtering (CF)	Phương pháp gợi ý dựa trên hành vi tập thể — gợi ý các mục mà những người dùng tương tự đã thích.
Hệ gợi ý lai	Hybrid Recommendation	Kết hợp Content-Based và Collaborative để phát huy ưu điểm của cả hai.
Khởi động lạnh	Cold Start	Vấn đề khi không có đủ dữ liệu để hệ thống đưa ra gợi ý — xảy ra với user mới hoặc sản phẩm mới.
TF-IDF	Term Frequency – Inverse Document Frequency	Kỹ thuật biểu diễn văn bản thành vector số, tính tần suất từ kết hợp với độ phổ biến của từ trên toàn bộ tập tài liệu.
Cosine Similarity	—	Độ đo độ tương đồng giữa hai vector, tính bằng tỉ số tích vô hướng và tích độ dài; giá trị nằm trong khoảng [0, 1].
Ma trận User-Item	User-Item Matrix	Ma trận hai chiều mô tả tương tác giữa user và item (số lượng mua, rating...).
Ma trận thưa	Sparse Matrix	Ma trận có nhiều ô giá trị 0 — phổ biến trong recommendation vì mỗi user chỉ mua một phần nhỏ tổng

		sản phẩm.
Lazy Loading	—	Kỹ thuật trì hoãn việc khởi tạo đối tượng đến khi thực sự cần dùng, giúp tăng tốc khởi động ứng dụng.
Soft Delete	—	Xóa logic — đánh dấu is_active=False thay vì xóa thật, giữ được lịch sử dữ liệu.
Hard Delete	—	Xóa thật bản ghi khỏi database.
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ giữa bảng database và class Python, giúp truy vấn dữ liệu bằng cú pháp ngôn ngữ lập trình.
MVC	Model-View-Controller	Mẫu thiết kế tách biệt dữ liệu (Model), giao diện (View), và logic xử lý (Controller).
3NF	Third Normal Form	Dạng chuẩn 3 của database — đảm bảo không có phụ thuộc bắc cầu giữa các cột phi khóa.
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản trên dữ liệu.
RBAC	Role-Based Access Control	Mô hình phân quyền dựa trên vai trò người dùng.
Vibe Engineering	—	Phương pháp phát triển phần mềm có sử dụng AI/GenAI làm trợ lý, kết hợp với review và chỉnh sửa của con người.
Prompt	—	Câu lệnh/yêu cầu gửi cho AI để nhận về phản hồi.
Path Traversal	—	Tấn công bảo mật cho phép kẻ tấn công truy cập file ngoài thư mục cho phép thông qua các ký tự ../.
SQL Injection	—	Tấn công chèn câu lệnh SQL độc hại qua các input của ứng dụng.
XSS	Cross-Site Scripting	Tấn công chèn mã JavaScript độc hại vào nội dung trang web.
CSRF	Cross-Site Request Forgery	Tấn công lừa người dùng thực hiện hành động không mong muốn trên trang web đã đăng nhập.

Phụ lục H — Tổng hợp các Prompt đã dùng trong dự án

Phụ lục này tổng hợp các prompt tiêu biểu mà nhóm đã dùng với AI/GenAI trong quá trình phát triển, theo từng giai đoạn dự án.

H.1. Prompt giai đoạn thiết kế

Prompt 1 — Phân tích yêu cầu:

"Tôi đang xây dựng một web bán thực phẩm (rau, thịt, gia vị) bằng Flask. Hãy giúp tôi liệt kê các chức năng cốt lõi cần có cho User (khách hàng) và Admin (quản trị viên), và phác thảo ERD cho hệ thống."

Prompt 2 — Sinh SQLAlchemy Model:

"Hãy viết các class model SQLAlchemy cho 5 bảng: User (có role user/admin, password hash), Category, Product (có giá, tồn kho, total_sold), Order (5 trạng thái), OrderItem (có snapshot giá). Code dùng Flask-SQLAlchemy, có comment tiếng Việt."

Prompt 3 — Vẽ ERD bằng Mermaid:

"Dựa trên 5 bảng trên, hãy vẽ ERD bằng cú pháp Mermaid erDiagram, ghi rõ kiểu dữ liệu và mối quan hệ giữa các bảng (1-N, N-N nếu có)."

H.2. Prompt giai đoạn lập trình

Prompt 4 — Sinh route Auth:

"Viết các route Flask cho /register, /login, /logout dùng Flask-Login. Validate: username ≥ 3 ký tự, email đúng định dạng, password ≥ 6 ký tự, password và confirm khớp. Mật khẩu băm bằng Werkzeug pbkdf2:sha256. Khi đăng nhập, admin được điều hướng về /admin, user thường về /."

Prompt 5 — Sinh class HybridRecommender:

"Viết class HybridRecommender bằng Python, dùng scikit-learn TfidfVectorizer cho Content-Based, và pandas pivot_table + cosine_similarity cho Collaborative Filtering. Có các hàm fit_content, fit_collaborative, get_similar_products, get_hybrid_recommendations. Phải có cơ chế xử lý Cold Start: khi user không có trong matrix thì gợi ý dựa trên content hoặc fallback Top Trending."

Prompt 6 — Sinh template Bootstrap:

"Viết template Jinja2 cho trang shop sản phẩm: hiển thị danh sách 12 sản phẩm/trang dưới dạng card Bootstrap 5, có sidebar lọc theo danh mục (checkbox) và slider giá, có ô tìm kiếm và dropdown sắp xếp. Có phân trang ở dưới."

Prompt 7 — Sinh JavaScript cho Chart.js:

"Viết JS để vẽ biểu đồ doanh thu theo tháng dùng Chart.js (line chart). Dữ liệu được Flask truyền qua biến revenue_data và revenue_labels (dùng filter | tojson). Tooltip hiển thị số tiền có định dạng VND."

H.3. Prompt giai đoạn debug

Prompt 8 — Debug OperationalError:

"Tôi gặp lỗi pymysql.err.OperationalError: (1049, 'Unknown database food_store_db') khi chạy python seed_data.py. Code của tôi có hàm ensure_database_exists() để tự tạo DB nhưng vẫn lỗi. Đây là code: [paste code]. Hãy phân tích nguyên nhân và đề xuất cách sửa."

Prompt 9 — Debug lỗi N+1 query:

"Khi tôi build Recommender với 2000 đơn hàng, SQLAlchemy gửi 2001 query SQL — rất chậm. Đây là code lấy orders: orders = Order.query.all() rồi for order in orders: for item in order.items: Làm sao tối ưu thành 1 query?"

Prompt 10 — Sửa lỗi giỏ hàng cộng dồn:

"Giỏ hàng của tôi không cộng dồn quantity khi user add cùng 1 sản phẩm 2 lần — lần 2 nó ghi đè lần 1. Đây là code: session['cart'][str(product_id)] = {'quantity': quantity}. Sửa giúp tôi."

H.4. Prompt giai đoạn kiểm thử

Prompt 11 — Sinh pytest fixture:

"Viết một pytest fixture tên client cho Flask Test Client. Phải dùng SQLite in-memory, tạo schema bằng db.create_all(), tắt CSRF (WTF_CSRF_ENABLED=False), và teardown bằng db.drop_all()."

Prompt 12 — Sinh test case Cold Start:

"Viết test case cho hàm get_hybrid_recommendations của HybridRecommender, tập trung vào tình huống Cold Start (user_id không có trong user_id_to_idx). Kết quả mong đợi: hệ thống tự ép alpha về 0, hoặc trả về fallback Top Trending."

Prompt 13 — Sinh test SQL Injection:

"Liệt kê 5 payload SQL Injection kinh điển. Sau đó viết pytest test case kiểm tra xem ứng dụng Flask của tôi có an toàn trước các payload này không (route login và route shop search)."

H.5. Prompt giai đoạn viết tài liệu

Prompt 14 — Sinh README:

"Viết README.md cho dự án FreshMart: phân giới thiệu, công nghệ sử dụng, hướng dẫn cài đặt 5 bước, các route chính, screenshot placeholder, và license MIT."

Prompt 15 — Sinh báo cáo Chương 1:

"Viết Chương 1 — Tổng quan đề tài cho báo cáo đồ án FreshMart. Bao gồm: bối cảnh, mục tiêu, đối tượng, phạm vi, ý nghĩa thực tiễn, user stories (8 US), kiến trúc tổng quan. Viết theo phong cách văn xuôi liền mạch tiếng Việt, không dùng quá nhiều bullet, độ dài khoảng 8 trang."

Tổng cộng, nhóm đã sử dụng **khoảng 40+ prompt** trong toàn bộ dự án, được tổ chức theo các giai đoạn trên. Mỗi prompt đều có chu trình **Prompt** → **Output** → **Edit** để đảm bảo chất lượng cuối cùng phù hợp với yêu cầu đặc thù của dự án.

